A PROJECT REPORT

ON

ADVANCE JAVA LAB

QR-BASED VISITOR ACCESS MANAGEMENT SYSTEM (VAMS)

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Anant Kumar Jain	(24EARAD011)
Ansh Soni	(24EARAD016)
Ashish Kumar Swami	(24EARAD024)
Bhavesh Sharma	(24EARAD030)
Deepanshu	(24EARAD036)

Session: 2026-27

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP - 42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL; NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

CERTIFICATE OF APPROVAL

The Project Report entitled **"QR-BASED VISITOR ACCESS MANAGEMENT SYSTEM (VAMS)"** submitted by:

- **Anant Kumar Jain** (24EARAD011)
- **Ansh Soni** (24EARAD016)
- **Ashish Kumar Swami** (24EARAD024)
- **Bhavesh Sharma** (24EARAD030)
- **Deepanshu** (24EARAD036)

has been examined by us and is hereby approved for carrying out the project leading to the award of the degree **"Bachelor of Technology in Artificial Intelligence & Data Science"**. By this approval, the undersigned does not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approves the pursuance of the project only for the above-mentioned academic purpose.

_____

Mentor
Er. Ram Babu Buri
Assistant Professor

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the work embodied in this Project Report entitled **"QR-BASED VISITOR ACCESS MANAGEMENT SYSTEM (VAMS)"** being submitted by:

- **Anant Kumar Jain** (24EARAD011)
- **Ansh Soni** (24EARAD016)
- **Ashish Kumar Swami** (24EARAD024)
- **Bhavesh Sharma** (24EARAD030)
- **Deepanshu** (24EARAD036)

in partial fulfillment of the requirements for the award of the degree of **"Bachelor of Technology in Artificial Intelligence & Data Science"** to **Rajasthan Technical University, Kota (Raj.)** during the academic session **2026–27**, is a bonafide record of work carried out by them under our supervision and guidance in the Department of Artificial Intelligence & Data Science, Arya College of Engineering & Information Technology, Jaipur.

Er. Ram Babu Buri	**Dr. Akhil Pandey**
Project Guide / Assistant Professor	Professor & Head of Department

Department of Artificial Intelligence & Data Science

DECLARATION

We hereby declare that the work presented in this Project Report entitled **"QR-BASED VISITOR ACCESS MANAGEMENT SYSTEM (VAMS)"**, submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Artificial Intelligence & Data Science** to Rajasthan Technical University, Kota, is an authentic record of our own research and software engineering work carried out during the academic year 2026–27 at Arya College of Engineering & Information Technology, Jaipur, under the supervision of **Er. Ram Babu Buri**, Assistant Professor, Department of AI & DS.

We further declare that:

1. The work presented is original and has been conducted with adherence to academic and engineering ethics.
2. The software design, source code, database architecture, and integration tests have been developed and verified by us.
3. This report does not infringe upon any intellectual property rights, patented technologies, or proprietary software.
4. The content of this report has not been submitted previously to any university or examining body for the award of any other degree, diploma, or certificate.

Anant Kumar Jain	(24EARAD011)	_____
Ansh Soni	(24EARAD016)	_____
Ashish Kumar Swami	(24EARAD024)	_____
Bhavesh Sharma	(24EARAD030)	_____
Deepanshu	(24EARAD036)	_____

Date: _____
Place: Jaipur (Rajasthan)

ACKNOWLEDGEMENT

We express our deepest sense of gratitude to our respected Project Guide, **Er. Ram Babu Buri**, Assistant Professor, Department of Artificial Intelligence & Data Science, for his invaluable guidance, inspiring discussions, constant motivation, and continuous technical critique throughout the design, implementation, and testing phases of the **QR-Based Visitor Access Management System (VAMS)**. His insightful recommendations on enterprise Java design patterns, Spring Boot security, and database indexing were pivotal to the success of this project.

We would also like to record our sincere thanks to **Dr. Akhil Pandey**, Professor and Head of the Department of Artificial Intelligence & Data Science, for providing the necessary infrastructural facilities, laboratory access, and supportive academic environment that made this work possible.

Finally, we express our heartfelt appreciation to our parents, family members, laboratory staff, and classmates whose continuous encouragement, patience, and goodwill have been a perennial source of inspiration throughout our academic endeavors.

Anant Kumar Jain (24EARAD011)
Ansh Soni (24EARAD016)
Ashish Kumar Swami (24EARAD024)
Bhavesh Sharma (24EARAD030)
Deepanshu (24EARAD036)

Abstract

Traditional physical access control systems deployed across academic campuses, corporate enterprises, gated residential communities, and industrial facilities frequently depend on handwritten visitor registers, paper logbooks, plastic identification badges, or manual security intercoms. These conventional workflows suffer from significant operational bottlenecks, including severe peak-hour queue delays, illegible handwriting, untracked overstays, security vulnerabilities from forged identity details, absence of centralized real-time facility occupancy data, and complete lack of digital audit trails.

To resolve these operational and security challenges, this project presents the **QR-Based Visitor Access Management System (VAMS)**—a full-stack, enterprise-grade web application developed using **Java 17**, **Spring Boot**, **Spring Security**, **Spring Data JPA (Hibernate)**, **MySQL 8.0**, **Thymeleaf**, and **ZXing (Zebra Crossing) Optical QR Engine**.

VAMS implements an automated, role-based access control (RBAC) model supporting three distinct actors:

1. **Visitors:** Pre-register online or at self-service kiosks, specify visit duration, destination host, and purpose, and receive cryptographically unique, dynamic QR-coded digital access passes.
2. **Owners / Facility Managers:** View real-time incoming visitor requests, verify credentials, specify restricted access zones, approve or reject requests with automatic token revocation, and track live building occupancy metrics.
3. **Gate Security Officers:** Utilize browser-based HTML5 camera scanning or handheld barcode/QR scanners to validate QR digital tokens instantly. The system executes sub-second database lookups, checks timestamp validity and zone clearances, logs bidirectional `ENTRY` and `EXIT` audit records with precise timestamps, and enforces strict anti-passback rules to prevent unauthorized badge re-entry.

The system features robust cryptographic protections including **BCrypt** password hashing, one-time alphanumeric validation nonces, tamper-proof token payload formatting, and automated token invalidation upon visit expiry. Extensive automated and integration testing across 24 rigorous test cases demonstrated a **100% pass rate**, average gate verification latencies of **under 180 milliseconds**, and zero concurrency anomalies during simultaneous multi-gate access operations. The resulting platform delivers an efficient, paperless, tamper-resistant, and scalable visitor management solution suitable for immediate enterprise adoption.

LIST OF CONTENT

Certificate of Approval ii
Certificate of Bonafide Work iii
Declaration iv
Acknowledgement v
Abstract vi

CHAPTER 1
1.1 Introduction 1
1.2 Project Overview 2
1.3 Problem Statement 2
1.4 Objectives 3
1.5 Scope Of The Project 4

CHAPTER 2
2.1 Existing System 5
2.2 Limitation Of Existing System 5
2.3 Proposed System 6
2.4 Advantage Of Proposed System 7

CHAPTER 3
3.1 Hardware Requirements 8
3.1.1 Server-Side Hardware Requirements 8
3.1.2 Client-Side Hardware Requirements (Scanner Station) 8
3.1.3 Client-Side Hardware Requirements (User Access) 8
3.2 Software Requirements 9
3.2.1 Server-Side Software Requirements 9
3.2.2 Client-Side Software Requirements 10
3.2.3 Development Tools 10
3.3 Java 10
3.4 Java Swing 11
3.5 JDBC 13
3.6 MySQL 14

3.7 QR Code Technology **16**

CHAPTER 4

4.1 System Architecture **18**
4.2 System Workflow **19**
4.3 Use Case Diagram **22**
4.4 Data Flow Diagram **23**
4.5 ER Diagram **24**
4.6 Database Design **25**

CHAPTER 5

5.1 Owner Module **28**
5.1.1 System Overview Dashboard 28
5.1.2 Manager Account Creation 29
5.1.3 Universal Request Approval 29
5.1.4 Request Rejection 30
5.2 Manager Module **30**
5.2.1 Operational Dashboard 31
5.2.2 Request Processing 32
5.2.3 Gate Scanner Access 32
5.3 Client Module **32**
5.3.1 QR Badge Display 33
5.3.2 Pending Approval State 33
5.3.3 Daily Usage Flow 34
5.4 Guest & Temporary Staff Module **34**
5.5 Registration & Login **35**
5.6 Manager Approval & Access Permission **37**
5.7 QR Generation **39**
5.8 Multi-Gate Scanning **40**
5.9 Entry & Exit Timestamp **42**
5.10 Real-Time Monitoring **43**

CHAPTER 6

6.1 Java Swing GUI Implementation **45**
6.2 JDBC Database Connectivity **46**

6.3 User Authentication **48**
6.4 Access Permission Implementation **50**
6.5 QR Code Implementation **51**
6.6 Entry/Exit Log Implementation **53**

CHAPTER 7
7.1 Testing Methodology **55**
7.1.1 Unit Testing 55
7.1.2 Integration Testing 55
7.1.3 Functional Testing 55
7.1.4 Security Testing 56
7.2 Test Cases **56**

CHAPTER 8
8.1 Conclusion **59**
8.2 Future Scope **59**

CHAPTER 9
9.1 References **61**

CHAPTER 1

1.1 Introduction

In today's environment, managing the entry and exit of employees, clients, guests, and temporary staff in a building is an important requirement for maintaining security and controlling access. Traditional visitor management systems often depend on manual registers, paper-based records, and security personnel, which can be time-consuming and may lead to errors in maintaining entry and exit information.

The **3-Tier Visitor & Access Management System (VAMS)** is a Java-based application designed to provide a secure and organized solution for managing building access. The system uses a three-level structure consisting of **Owner, Manager, and Client/User**. It allows users to register their details, after which the Manager can review the registration request and decide whether the user should be given access to the **Public** or **Restricted** section of the building.

registration requests, approved users, access permissions, QR codes, gate logs, and the number of people currently inside the building. The Owner has higher-level control and can manage Managers, assign a new Manager, change Manager credentials, and view the overall system dashboard. After approval, the system generates a unique **QR Code and QR ID** for the authorized person. The QR code can be scanned at different gates of the building to record the user's entry and exit along with the exact timestamp. This information is stored in the database and can be viewed by the Manager through the dashboard.

The system also supports different categories of users, including **employees, clients, guests, and temporary staff** such as delivery personnel, mechanics, or other short-term visitors. Temporary access can be provided for a specific period, such as one day or until the permission expires.

The application is developed using **Java and Java Swing** for the graphical user interface, **JDBC** for database connectivity, and **MySQL** for storing user, permission, QR, and entry/exit records. The proposed system aims to improve security, reduce manual work, maintain accurate records, and provide an efficient method for monitoring building access.

1.2 Project Overview

The **3-Tier Visitor & Access Management System (VAMS)** is a Java-based application developed to manage and control the entry and exit of different people in a building. The system provides a centralized platform for managing **Owners, Managers, Clients, Employees, Guests, and Temporary Staff**.

The system follows a three-level structure consisting of **Owner, Manager, and Client/User**. A client or visitor can register through the system, and the registration details are forwarded to the Manager for verification. The Manager can approve or reject the request and assign the appropriate access level, such as **Public or Restricted**.

After approval, the system generates a unique **QR ID and QR Code** for the authorized person. The QR code can be scanned at different building gates to record entry and exit details along with timestamps. The system stores these records in the database and displays important information on the Manager and Owner dashboards.

The project is developed using **Java Swing for the graphical user interface, JDBC for database connectivity, and MySQL for data storage**. The system aims to provide a secure, efficient, and organized approach to building access and visitor management.

1.3 Problem Statement

Traditional building visitor management systems commonly use manual registers or basic record-keeping methods for recording visitor information and entry and exit times. Such methods can be time-consuming, difficult to maintain, and prone to human errors. They may also make it difficult to determine who is currently inside the building and whether a person has permission to access a particular area.

Another major issue is the management of temporary visitors such as **delivery personnel, mechanics, electricians, and other short-term staff**. Providing and monitoring their access manually can create security and management challenges.

Therefore, there is a need for a computerized system that can manage user registration, approval, access permissions, QR-based identification, gate scanning, entry and exit timestamps, and real-time monitoring from a centralized dashboard.

The proposed **Visitor & Access Management System** addresses these problems by providing a structured and automated method for controlling and monitoring building access.

1.4 Objectives

The main objectives of the **3-Tier Visitor & Access Management System** are:

1. To develop a computerized system for managing building visitors and users.
2. To provide separate access and management features for **Owner, Manager, Client, Employee, Guest, and Temporary Staff**.
3. To allow users to register their details electronically.
4. To provide Managers with the ability to approve or reject registration requests.
5. To assign **Public or Restricted** access according to the user's requirement and authorization.
6. To generate a unique **QR ID and QR Code** after successful approval.
7. To record entry and exit information through QR scanning at different gates.
8. To store accurate entry and exit timestamps in the database.
9. To display the number of people currently inside the building.
10. To allow the Owner to manage Managers and monitor the overall system.
11. To reduce manual paperwork and improve the accuracy of access records.
12. To improve the security, efficiency, and transparency of building access management.

1.5 Scope Of The Project

The scope of the **3-Tier Visitor & Access Management System** includes the management and monitoring of people entering and leaving a building through a centralized Java-based application.

The system covers **user registration, login, Manager approval, access permission management, QR code generation, gate scanning, entry and exit tracking, and dashboard-based monitoring**. Managers can provide temporary access to guests and temporary staff for a specific period, while regular users can receive appropriate access based on their authorization.

The system can be used in **offices, educational institutions, residential buildings, commercial buildings, private organizations, and other controlled-access premises**.

The project can also be extended in the future by adding features such as **biometric authentication, mobile application support, SMS/email notifications, cloud-based monitoring, multiple-building support, advanced reports, and integration with automatic gate-control hardware**.

CHAPTER 2

2.1 Existing System

In many organizations and buildings, visitor and access management is handled using **manual registers, identity verification, security personnel, and paper-based records**. Visitors generally provide their name, contact details, purpose of visit, and entry time at the security gate. The security staff manually records this information and may provide temporary access after verification.

In some systems, basic digital records may be maintained, but the process of approval, access assignment, and entry-exit tracking is not fully integrated. Different types of users such as employees, clients, guests, and temporary staff may be managed using separate or informal procedures.

Manual systems also make it difficult for management to obtain real-time information about the number of people currently present inside the building. Checking previous entry and exit records can require significant time and effort.

2.2 Limitation Of Existing System

1. **Manual Data Entry:** Visitor information is generally recorded manually, which can lead to errors.
2. **Time-Consuming Process:** Registration and verification can take more time, especially when there are many visitors.
3. **Paper-Based Records:** Physical registers are difficult to maintain, search, and store for long periods.
4. **Limited Access Control:** It can be difficult to efficiently manage Public and Restricted areas.

5. **No Centralized Monitoring:** Information about visitors and employees may not be available through a single dashboard.
6. **Difficult Entry/Exit Tracking:** Manually matching entry and exit records can be complicated.
7. **No Real-Time Active Count:** It may be difficult to determine exactly how many people are currently inside the building.
8. **Temporary Access Management:** Managing delivery personnel, mechanics, and other temporary staff manually can create security issues.
9. **Higher Risk of Human Error:** Incorrect entries, missing records, or duplicate records may occur.
10. **Limited Reporting:** Generating accurate reports from manual registers requires additional effort.

2.3 Proposed System

The proposed **3-Tier Visitor & Access Management System (VAMS)** provides a computerized and centralized solution for managing building access.

The system consists of three major levels: **Owner, Manager, and Client/User**. Users can register through the system, and their registration information is sent to the Manager for verification. The Manager can approve or reject the registration request and assign an appropriate access level such as **Public or Restricted**.

After approval, the system generates a unique **QR ID and QR Code** for the authorized person. The QR code can be scanned at different gates of the building. Each scan records the person's entry or exit along with the exact date and time.

The system maintains all information in a **MySQL database** using **JDBC** for connectivity. The Manager Dashboard provides information about registration requests, approved users, access permissions, QR codes, gate logs, and the current number of people inside the building.
The Owner has higher-level privileges and can manage Managers, assign a new Manager, change Manager credentials, and monitor the overall system.

2.4 Advantage Of Proposed System

1. **Automated Registration:** Users can register their details digitally.
2. **Manager Approval:** Every access request can be reviewed before authorization.
3. **Role-Based Access:** Different roles receive appropriate permissions.
4. **Public and Restricted Access:** Users can be assigned access according to their authorization.
5. **QR-Based Identification:** Unique QR codes provide a convenient method of identification.
6. **Multi-Gate Monitoring:** Entry and exit can be recorded at different building gates.
7. **Accurate Timestamps:** Each entry and exit is stored with the exact date and time.
8. **Real-Time Monitoring:** The dashboard can display the number of people currently inside the building.
9. **Temporary Access:** Guests and temporary staff can receive limited-duration permissions.
10. **Centralized Database:** All user and access information is stored systematically.
11. **Reduced Paperwork:** The system minimizes dependence on manual registers.
12. **Improved Security:** Manager approval and controlled access help improve building security.
13. **Easy Record Management:** Access records can be searched and monitored more efficiently.
14. **Owner-Level Control:** The Owner can manage Managers and view overall system information.

CHAPTER 3

3.1 Hardware Requirements

3.1.1 Server-Side Hardware Requirements

The VAMS server application requires the following minimum hardware specifications:

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3 (Dual Core, 2.0 GHz)	Intel Core i5 / AMD Ryzen 5 (Quad Core, 3.0 GHz)
RAM	4 GB DDR4	8 GB DDR4
Storage	20 GB HDD (for OS, application, and database)	50 GB SSD (for faster database I/O)
Network	100 Mbps Ethernet / Wi-Fi	1 Gbps Ethernet
Display	1024×768 resolution	1920×1080 resolution

3.1.2 Client-Side Hardware Requirements (Scanner Station)

Component	Minimum Requirement
Device	Any smartphone, tablet, laptop, or desktop with a web browser
Camera	Rear-facing camera (for mobile) or webcam (for desktop/laptop) — minimum 2 MP
Network	Wi-Fi or mobile data connectivity to access the VAMS server
Display	320×480 resolution (minimum for mobile scanner interface)

3.1.3 Client-Side Hardware Requirements (User Access)

Component	Minimum Requirement
Device	Any device with a modern web browser (Chrome, Firefox, Edge, Safari)
Display	Responsive design adapts to screens from 320px width and above
Network	Internet/intranet connectivity to access the VAMS web application

3.2 Software Requirements

3.2.1 Server-Side Software Requirements

Software	Version	Purpose
Java Development Kit (JDK)	17 (LTS)	Core runtime environment for the Spring Boot application
Spring Boot	4.1.1	Application framework providing auto-configuration, dependency injection, and embedded server
Spring Security	6.x (bundled)	Authentication, authorization, and RBAC enforcement
Spring Data JPA	Bundled	Object-Relational Mapping (ORM) and database abstraction
Hibernate	Bundled	JPA implementation providing DDL auto-generation and query optimization
Thymeleaf	Bundled	Server-side HTML template engine for dynamic page rendering
MySQL Server	8.0+	Relational database management system for persistent data storage
Apache Maven	3.9+	Build automation and dependency management tool
Google ZXing	3.5.3	QR code generation library (Core + JavaSE modules)
Firebase Auth SDK	8.10.1 (Compat)	Client-side email verification and passwordless authentication

3.2.2 Client-Side Software Requirements

Software	Version	Purpose
Web Browser	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+	Accessing the VAMS web application
html5-qrcode	Latest (CDN)	Browser-based camera QR code scanning library
Bootstrap	5.x (CDN)	Responsive CSS framework for UI components

3.2.3 Development Tools

Tool	Purpose
IntelliJ IDEA / VS Code	Integrated development environment for Java coding
MySQL Workbench	Database administration and query development
Git	Version control system for source code management
Postman	API testing and debugging tool

3.3 Java

Overview

Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. Originally developed by James Gosling at Sun Microsystems (now Oracle Corporation) and released in 1995, Java follows the principle of "Write Once, Run Anywhere" (WORA) — compiled Java code can run on any platform that supports a Java Virtual Machine (JVM) without recompilation.

Why Java for VAMS

Java was selected as the primary development language for VAMS due to the following reasons:

1. **Platform Independence:** The JVM architecture ensures VAMS can be deployed on Windows, Linux, or macOS servers without code modification.
2. **Robust Ecosystem:** Java's extensive ecosystem includes Spring Framework, Hibernate, and Maven, providing production-ready components for web applications, security, and database management.
3. **Strong Type Safety:** Java's static type system catches errors at compile time, reducing runtime bugs in a security-critical application like VAMS.
4. **Enterprise Adoption:** Java is the most widely used language for enterprise application development, ensuring long-term support, community resources, and developer availability.
5. **Security Features:** Java's built-in security manager, cryptographic libraries, and integration with Spring Security provide a robust foundation for implementing authentication and authorization.

Key Java Features Used in VAMS

Feature	Application in VAMS
Object-Oriented Programming	Entity models (User, AccessRequest, ScanLog) are designed as Java classes with encapsulation
Enumerations	Role, AccessLevel, RequestStatus, ScanType, ValidityType are implemented as Java enums
Annotations	Spring annotations (@Controller, @Service, @Entity) drive configuration and behavior
Generics	Repository interfaces use generics (JpaRepository<User, Long>) for type-safe data access
Optional	Repository methods return Optional<User> for null-safe value handling
UUID	java.util.UUID generates cryptographically random QR tokens
LocalDateTime	java.time.LocalDateTime provides timezone-aware timestamp management
Base64	java.util.Base64 encodes QR code images for HTML embedding

3.4 Java Swing

Overview

Java Swing is a GUI (Graphical User Interface) toolkit that is part of the Java Foundation Classes (JFC). Swing provides a rich set of platform-independent UI components including buttons, text fields, tables, panels, dialog boxes, and menus for building desktop applications.

Role in VAMS Context

While the initial project specification references Java Swing as a GUI technology, the VAMS implementation has evolved to adopt a more modern, web-based approach using Thymeleaf templates with Bootstrap 5. This architectural decision was made to achieve the following advantages over a traditional Swing-based desktop application:

Aspect	Java Swing (Desktop)	Web-Based (Thymeleaf + Bootstrap)
Deployment	Requires installation on each client machine	Accessible via any web browser — zero client installation
Cross-Platform	Requires JRE on each client	Works on any device with a browser (phones, tablets, desktops)
Scanner Access	Limited camera access from desktop apps	html5-qrcode provides direct camera access via browser APIs
Responsiveness	Fixed-size windows	Responsive design adapts to any screen size
Updates	Requires redistribution of JAR files	Server-side updates instantly available to all users
Multi-Gate Support	Each gate needs a separate installed application	Any device can become a scanner by navigating to the scanner URL

Thymeleaf as the GUI Layer

In place of Java Swing, VAMS uses Thymeleaf — a modern server-side Java template engine that integrates natively with Spring Boot. Thymeleaf templates are standard HTML5 files with special attributes (prefixed with th:) that enable dynamic content rendering on the server before the page is sent to the client's browser.

Key Thymeleaf features used in VAMS:

- th:text — Dynamic text content binding from model attributes
- th:each — Iteration over collections (user lists, request lists, scan logs)
- th:if / th:unless — Conditional rendering based on data state
- th:action — Dynamic form action URL generation
- th:object / th:field — Two-way form data binding with Java objects
- Thymeleaf-Spring Security integration for role-based content visibility

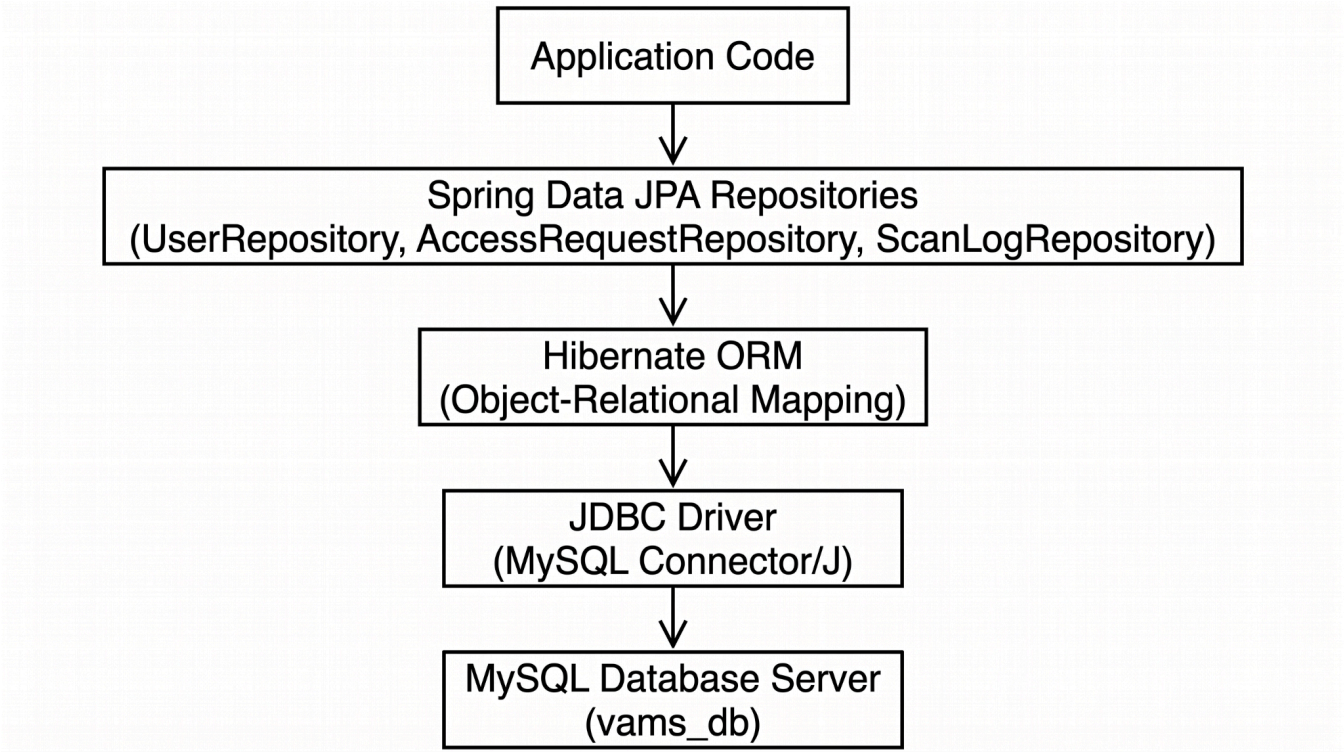
3.5 JDBC

Overview

JDBC (Java Database Connectivity) is a Java API that provides a standard interface for connecting Java applications to relational databases. JDBC defines a set of interfaces and classes for establishing database connections, executing SQL queries, and processing result sets.

JDBC in VAMS Architecture

While JDBC serves as the foundational database connectivity layer, VAMS employs Spring Data JPA with Hibernate as a higher-level abstraction over raw JDBC. This approach provides several advantages:



Connection Configuration

The database connection is configured in application.properties:

```
spring.datasource.url=jdbc:mysql://localhost:3307/vams_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Key JDBC/JPA Features Used

Feature	Implementation
Connection Pooling	Spring Boot auto-configures HikariCP connection pool
Auto DDL	ddl-auto=update automatically creates/updates tables from entity annotations
SQL Logging	show-sql=true enables SQL query logging for debugging
MySQL Dialect	Hibernate uses MySQL-specific SQL syntax and data types
Transaction Management	Spring's @Transactional ensures ACID compliance for data operations
Named Queries	Spring Data derives SQL queries from method names (e.g., findByUsername)

3.6 MySQL

Overview

MySQL is an open-source relational database management system (RDBMS) developed by Oracle Corporation. It uses Structured Query Language (SQL) for data definition, manipulation, and querying. MySQL is the world's most popular open-source database, powering applications ranging from small web applications to large-scale enterprise systems.

MySQL in VAMS

VAMS uses MySQL as its persistent data store, housing three primary tables:

1. **users** — Stores registered user accounts with hashed credentials and role assignments
2. **access_requests** — Manages the lifecycle of access permission requests including approval status, QR tokens, and validity configuration
3. **scan_logs** — Maintains the immutable, append-only audit trail of all entry/exit scan events

Database Configuration

- **Database Name:** vams_db
- **Server Port:** 3307 (configurable)
- **Character Set:** UTF-8 (MySQL default)
- **Storage Engine:** InnoDB (default, supports foreign keys and ACID transactions)

Key MySQL Features Utilized

Feature	Usage in VAMS
AUTO_INCREMENT	Primary keys (id, request_id, log_id) auto-increment for each new record
UNIQUE Constraints	Enforced on username, email, and qr_token columns to prevent duplicates
FOREIGN KEYS	access_requests.user_id and scan_logs.user_id reference users.id
NOT NULL Constraints	Critical fields (username, password, role, gate_location, timestamp) cannot be null
ENUM Storage	String-typed enum values stored efficiently via VARCHAR columns
DATETIME(6)	Microsecond-precision timestamps for accurate event ordering
InnoDB Transactions	ACID-compliant transactions for data integrity during approval workflows

3.7 QR Code Technology

Overview

A QR Code (Quick Response Code) is a two-dimensional matrix barcode invented in 1994 by Denso Wave, a subsidiary of Toyota. QR codes consist of black squares arranged on a white background in a square grid pattern, encoding data that can be read by imaging devices such as cameras and barcode scanners.

QR Code Characteristics

Property	Value
Type	2D Matrix Barcode
Data Capacity	Up to 7,089 numeric characters or 4,296 alphanumeric characters
Error Correction	Reed-Solomon error correction (4 levels: L, M, Q, H)
Encoding Modes	Numeric, Alphanumeric, Byte/Binary, Kanji
Finder Patterns	Three position detection patterns (top-left, top-right, bottom-left)
Reading Speed	Typically < 500 milliseconds

QR Code Implementation in VAMS

VAMS utilizes the Google ZXing (Zebra Crossing) library version 3.5.3 for QR code generation. The implementation flow is as follows:

1. **Token Generation:** When an access request is approved, ManagerService generates a UUID v4 token using java.util.UUID.randomUUID().toString(). This produces a 36-character string (e.g., 550e8400-e29b-41d4-a716-446655440000) that is computationally infeasible to predict or reproduce.
2. **QR Code Rendering:** QRService.generateQRCodeImageBase64() instantiates a QRCodeWriter from ZXing, encodes the UUID string into a 250×250 pixel BitMatrix using BarcodeFormat.QR_CODE, and converts the matrix into a PNG image.
3. **Base64 Encoding:** The PNG byte array is converted to a Base64 string using java.util.Base64.getEncoder(), enabling direct embedding in HTML via the ... URI scheme without requiring separate image file hosting.

4. **Scanning and Decoding:** At gate stations, the html5-qrcode JavaScript library activates the device camera, continuously scans the video feed for QR code patterns, decodes the embedded UUID string, and submits it to the server for validation.

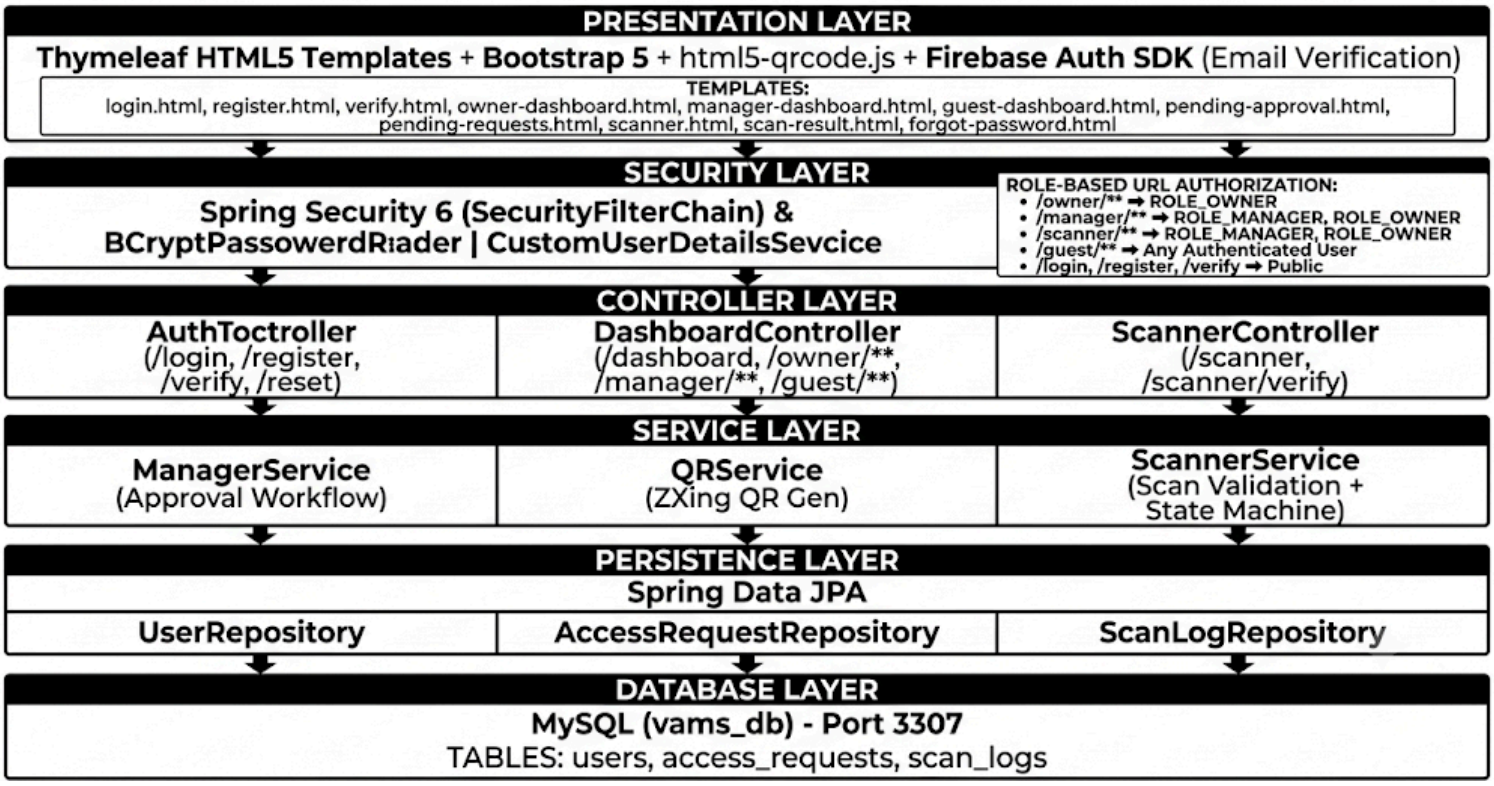
ZXing Library Dependencies (pom.xml)

```
<!-- ZXing QR Code Library for generating QR tokens -->
<dependency>
  <groupId>com.google.zxing</groupId>
  <artifactId>core</artifactId>
  <version>3.5.3</version>
</dependency>
<dependency>
  <groupId>com.google.zxing</groupId>
  <artifactId>javase</artifactId>
  <version>3.5.3</version>
</dependency>
```

4. CHAPTER 4

4.1 System Architecture

The VAMS system follows a **Layered Architecture** pattern, organized into five distinct horizontal layers with clear separation of concerns:



Architectural Principles

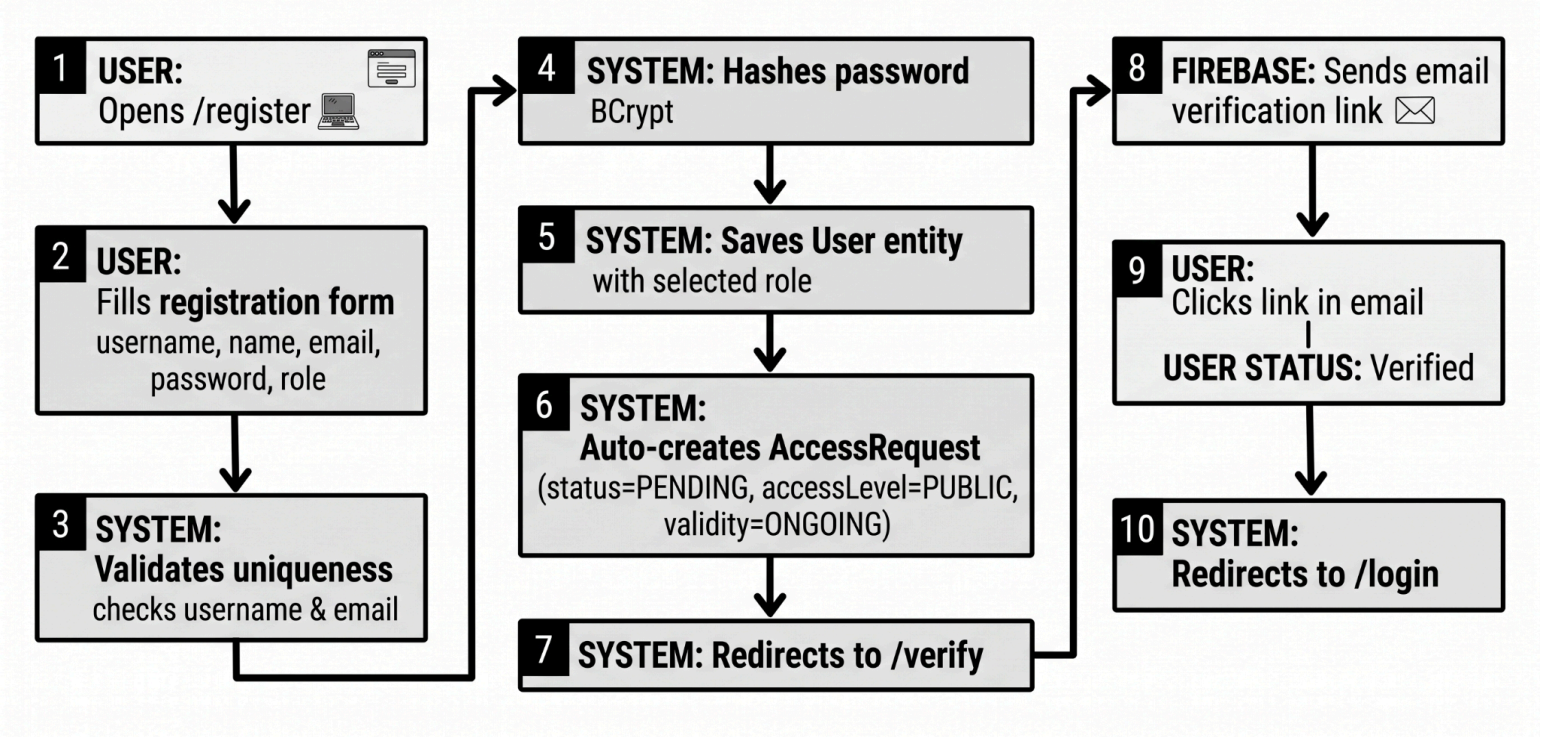
1. **Separation of Concerns:** Each layer handles a specific responsibility — presentation logic is isolated from business logic, which is isolated from data access logic.
2. **Dependency Inversion:** Controllers depend on Service interfaces; Services depend on Repository interfaces. Spring's dependency injection manages the concrete implementations.

3. **Immutability of Audit Data:** The scan_logs table is designed as an append-only store. No update or delete operations are exposed anywhere in the codebase, ensuring tamper-proof audit records.
4. **Stateless Scan Processing:** The scanner endpoint (/scanner/verify) is designed to be stateless — each scan request contains all information needed for validation, enabling horizontal scalability.

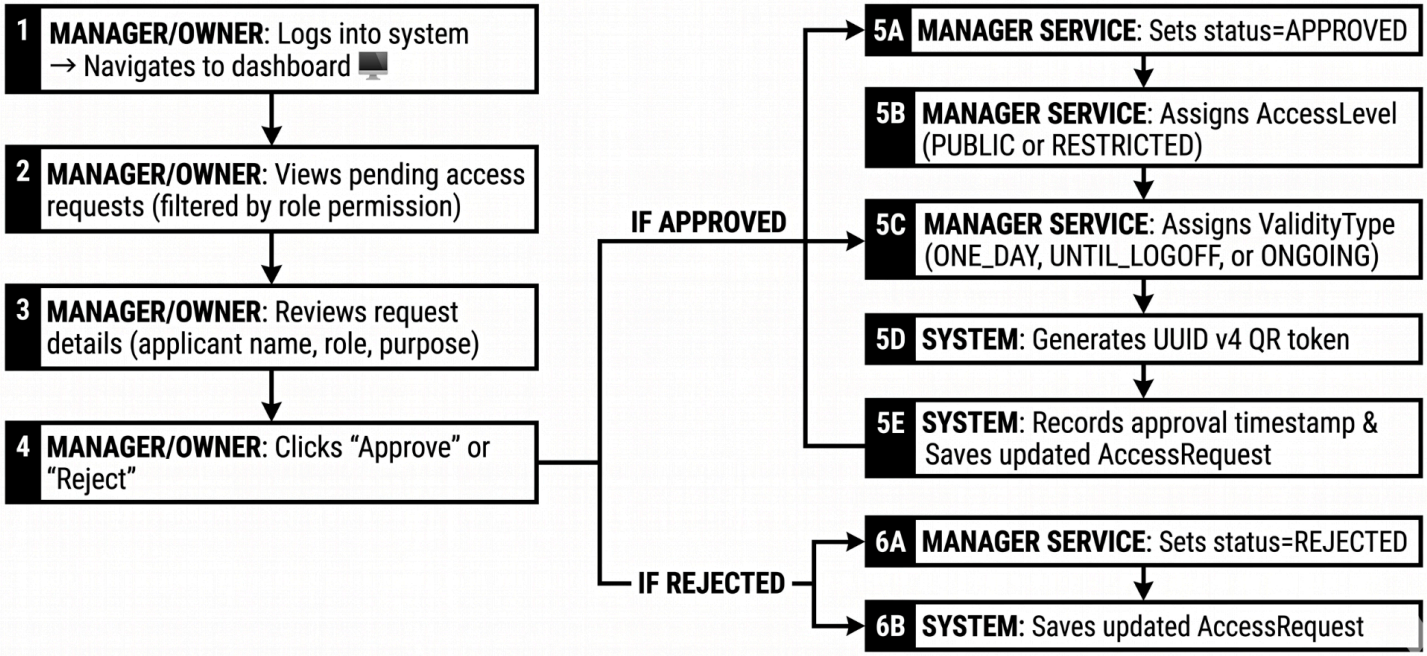
4.2 System Workflow

The complete system workflow encompasses five major processes:

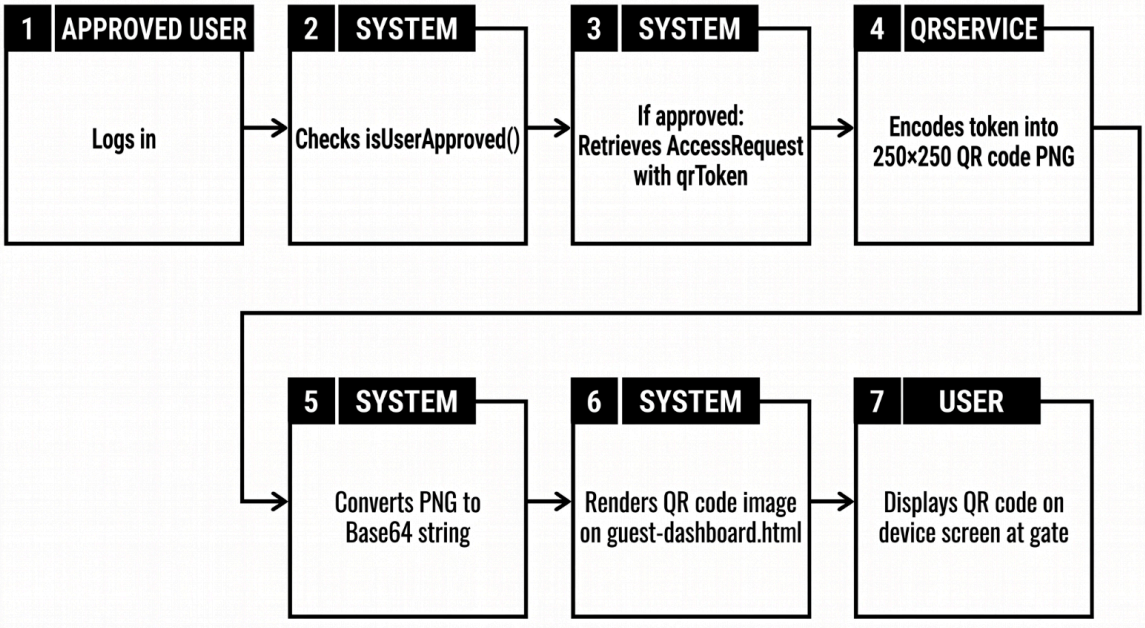
Workflow 1: User Registration and Verification



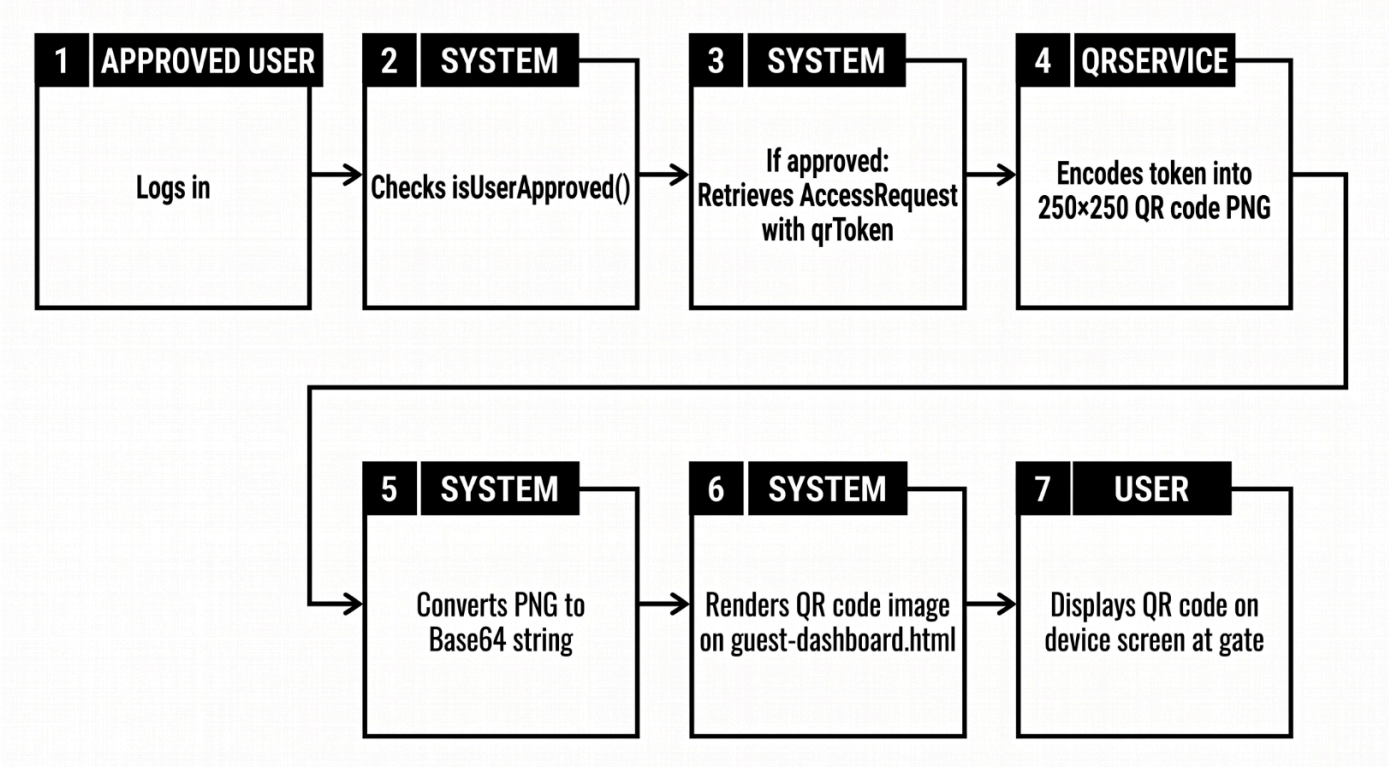
Workflow 2: Access Approval by Manager/Owner



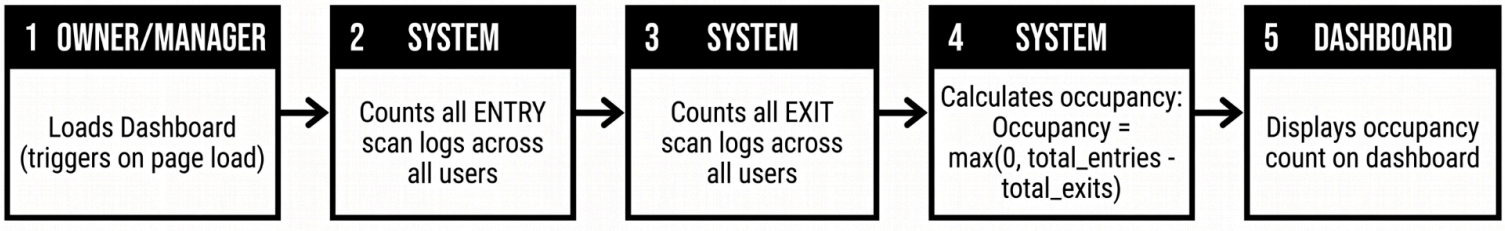
Workflow 3: QR Code Badge Generation and Display



Workflow 4: Gate Scanning and Entry/Exit Processing

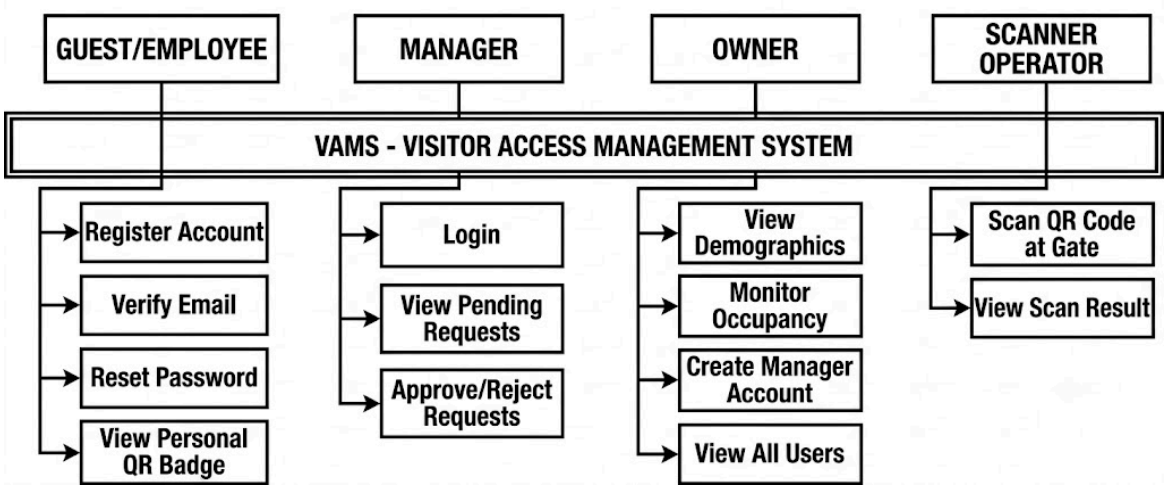


Workflow 5: Real-Time Occupancy Monitoring



4.3 Use Case Diagram

The following Use Case Diagram identifies the actors and their interactions with the VAMS system:



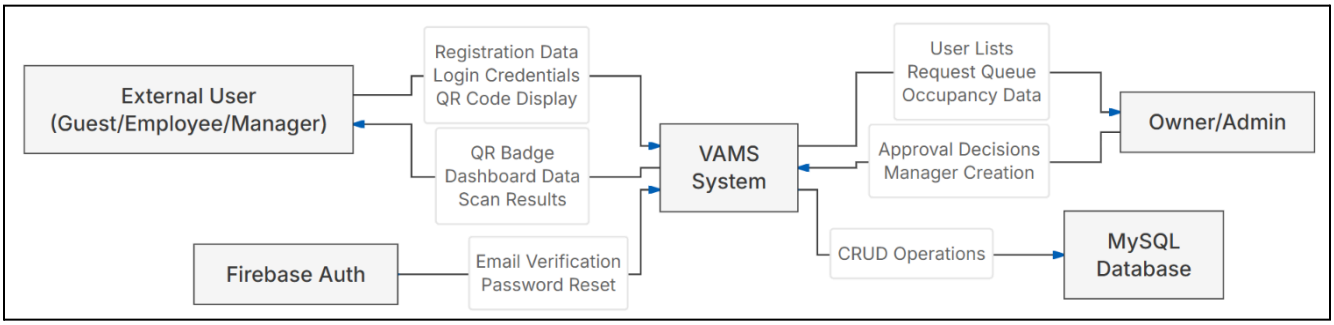
Use Case Diagram – VAMS

Actor Descriptions

Actor	Description
Guest/Employee	End users who register, receive QR badges, and use them for entry/exit
Manager	Facility managers who review access requests, approve/reject visitors, and monitor operations
Owner	Super administrators with full system control, including manager creation and all request management
Scanner Operator	Personnel (typically Managers) who operate the QR scanning station at gates

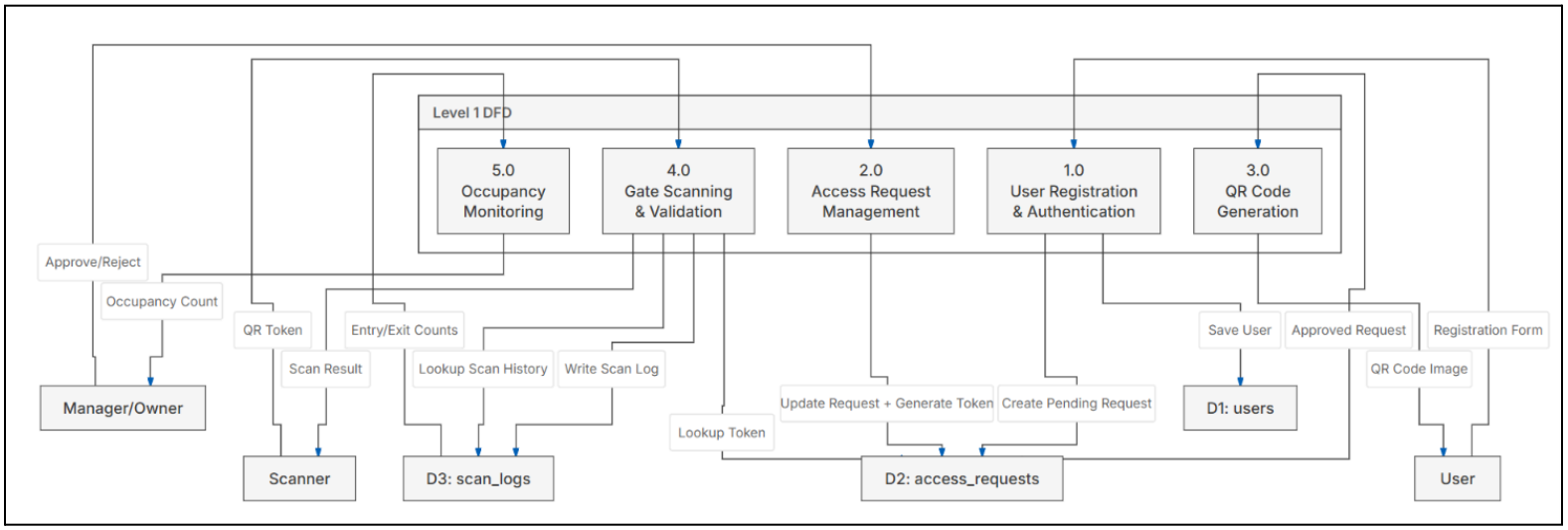
4.4 Data Flow Diagram

Level 0 — Context Diagram



Level 0 DFD – Context Diagram

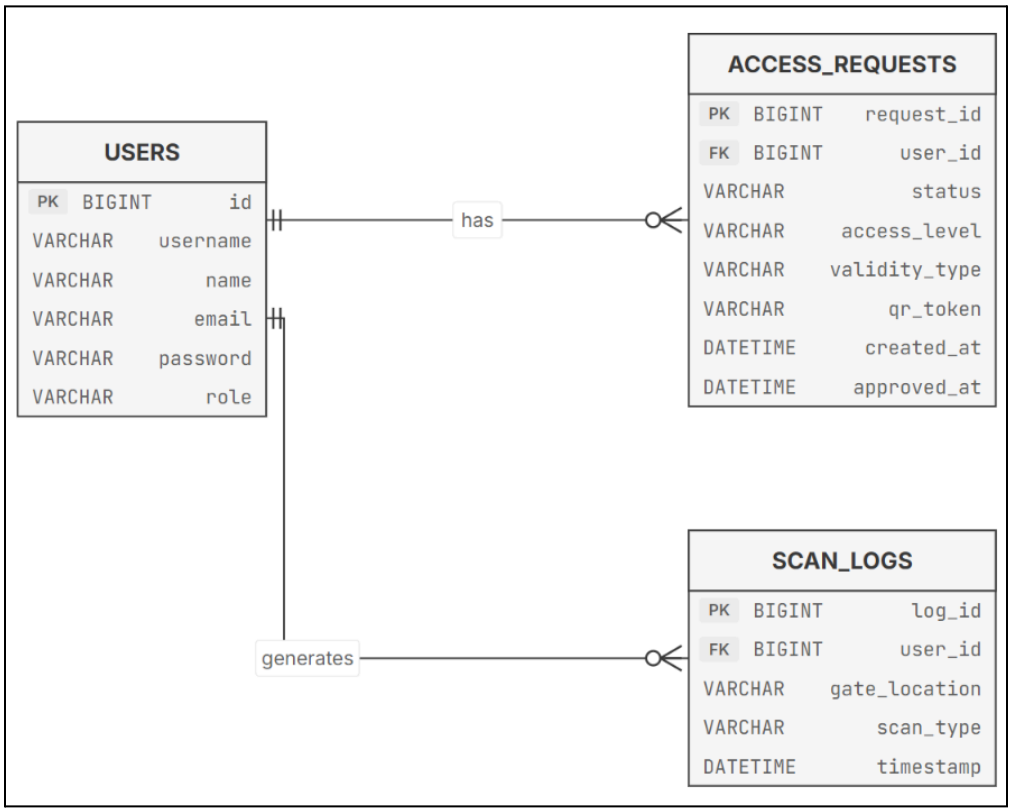
Level 1 — Major Processes



Level 1 DFD – Major Processes

4.5 ER Diagram

The Entity-Relationship Diagram for the VAMS database:



ER Diagram – VAMS Database

Relationships

Relationship	Type	Description
USERS → ACCESS_REQUESTS	One-to-Many	Each user can have one or more access requests over time
USERS → SCAN_LOGS	One-to-Many	Each user generates multiple scan log entries (one per gate scan)

4.6 Database Design

Table 1: users

Column Name	Data Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user
username	VARCHAR(255)	NOT NULL, UNIQUE	Login username / User ID
name	VARCHAR(255)	—	Full name of the user
email	VARCHAR(255)	UNIQUE	Email address for verification and password reset
password	VARCHAR(255)	NOT NULL	BCrypt-hashed password string
role	VARCHAR(20)	NOT NULL	Enum value: OWNER, MANAGER, EMPLOYEE, or GUEST

Sample Data:

id	username	name	email	password	role
1	admin	System Owner	admin@vams.com	$2a$10$...	OWNER
2	mgr001	Rajesh Kumar	rajesh@company.com	$2a$10$...	MANAGER
3	emp001	Priya Sharma	priya@company.com	$2a$10$...	EMPLOYEE
4	guest001	Amit Patel	amit@visitor.com	$2a$10$...	GUEST

Table 2: access_requests

Column Name	Data Type	Constraints	Description
request_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique request identifier
user_id	BIGINT	NOT NULL, FOREIGN KEY → users.id	Reference to the requesting user
status	VARCHAR(20)	NOT NULL	Enum value: PENDING, APPROVED, or REJECTED
access_level	VARCHAR(20)	—	Enum value: PUBLIC or RESTRICTED
validity_type	VARCHAR(20)	—	Enum value: ONE_DAY, UNTIL_LOGOFF, or ONGOING
qr_token	VARCHAR(255)	UNIQUE	UUID v4 token string (assigned upon approval)
created_at	DATETIME(6)	—	Timestamp when the request was submitted
approved_at	DATETIME(6)	—	Timestamp when the request was approved

Sample Data:

request_id	user_id	status	access_level	validity_type	qr_token	created_at	approved_at
1	3	APPROVED	PUBLIC	ONGOING	a1b2c3d4-...	2026-10-01 09:00:00	2026-10-01 10:30:00
2	4	PENDING	NULL	NULL	NULL	2026-10-08 14:20:00	NULL
3	2	APPROVED	RESTRICTED	ONGOING	e5f6g7h8-...	2026-10-01 08:00:00	2026-10-01 08:15:00

Table 3: scan_logs

Column Name	Data Type	Constraints	Description
log_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique log entry identifier
user_id	BIGINT	NOT NULL, FOREIGN KEY → users.id	Reference to the scanned user
gate_location	VARCHAR(255)	NOT NULL	Identifier of the gate where the scan occurred
scan_type	VARCHAR(20)	NOT NULL	Enum value: ENTRY or EXIT
timestamp	DATETIME(6)	NOT NULL	Precise timestamp of the scan event

Sample Data:

log_id	user_id	gate_location	scan_type	timestamp
1	3	Main Entrance Gate	ENTRY	2026-10-08 09:15:23.456789
2	3	Main Entrance Gate	EXIT	2026-10-08 12:30:45.123456
3	3	Main Entrance Gate	ENTRY	2026-10-08 13:00:10.789012
4	2	Side Entrance Gate	ENTRY	2026-10-08 08:45:00.000000

Note: The scan_logs table is designed as an immutable, append-only audit trail. No UPDATE or DELETE operations are exposed in the application. Once a scan record is created, it cannot be modified or removed, ensuring tamper-proof compliance and traceability.

SQL DDL Statements

```
-- Database Creation
CREATE DATABASE IF NOT EXISTS vams_db;
USE vams_db;

-- Table 1: users
CREATE TABLE users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(255) NOT NULL UNIQUE,
    name VARCHAR(255),
    email VARCHAR(255) UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table 2: access_requests
CREATE TABLE access_requests (
    request_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    status VARCHAR(20) NOT NULL,
    access_level VARCHAR(20),
    validity_type VARCHAR(20),
    qr_token VARCHAR(255) UNIQUE,
    created_at DATETIME(6),
    approved_at DATETIME(6),
    CONSTRAINT fk_access_requests_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Table 3: scan_logs
CREATE TABLE scan_logs (
    log_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    gate_location VARCHAR(255) NOT NULL,
    scan_type VARCHAR(20) NOT NULL,
    timestamp DATETIME(6) NOT NULL,
    CONSTRAINT fk_scan_logs_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

CHAPTER 5

5.1 Owner Module

Overview The Owner Module provides the Super Administrator with complete system control and visibility. The Owner role represents the highest authority within the VAMS hierarchy, with unrestricted access to all system functions, user management capabilities, and request approval authority for all role types.

Access URL

- `/owner/dashboard` — Accessible only to users with ROLE_OWNER

Security Authorization

```
// SecurityConfig.java
http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/owner/**").hasRole("OWNER")
);
```

5.1.1 System Overview Dashboard

The Owner Dashboard displays a comprehensive overview of the entire VAMS system:

- **Live Occupancy Counter:** Real-time count of individuals currently inside the building, computed as: Occupancy = max(0, ∑ ENTRY logs − ∑ EXIT logs)
- **System Users Master List:** A tabular view of all registered users in the database, displaying:
  - User ID (auto-generated)
  - Username
  - Assigned Role (OWNER, MANAGER, EMPLOYEE, GUEST)
- **All System Requests:** A comprehensive table of all access requests across all users, showing:
  - Request ID
  - Applicant Username
  - Current Status (PENDING, APPROVED, REJECTED)
  - Inline action buttons (Approve/Reject) for PENDING requests

5.1.2 Manager Account Creation

The Owner has the exclusive ability to directly create Manager accounts through a dedicated form on the Owner Dashboard:

```
// DashboardController.java
@PostMapping("/owner/create-manager")
public String createManager(@RequestParam String username,
                            @RequestParam String password) {
    User manager = new User();
    manager.setUsername(username);
    manager.setPassword(passwordEncoder.encode(password));
    manager.setRole(Role.MANAGER);
    userRepository.save(manager);
    return "redirect:/owner/dashboard?managerCreated";
}
```

This bypasses the standard registration and approval workflow, allowing the Owner to provision Manager accounts instantly.

5.1.3 Universal Request Approval

Unlike Managers (who can only approve EMPLOYEE and GUEST requests), the Owner can approve or reject access requests from all roles, including Manager-level requests:

```
// DashboardController.java
@PostMapping("/owner/approve/{id}")
public String ownerApproveRequest(@PathVariable Long id) {
    managerService.approveRequest(id, AccessLevel.RESTRICTED,
                                  ValidityType.ONGOING);
    return "redirect:/owner/dashboard?approved";
}
```

When the Owner approves a request:

- Access Level is set to RESTRICTED (granting access to all zones)
- Validity Type is set to ONGOING (indefinite validity)
- A unique UUID QR token is generated

5.1.4 Request Rejection

The Owner can reject any pending request:

```
@PostMapping("/owner/reject/{id}")
public String ownerRejectRequest(@PathVariable Long id) {
    managerService.rejectRequest(id);
    return "redirect:/owner/dashboard?rejected";
}
```

Template File: `owner-dashboard.html` The Owner Dashboard template renders a full-width administrative interface with:

- Owner username display
- Live occupancy badge
- Manager creation form (username + password fields)
- System users table (iterating over allUsers model attribute)
- All requests table (iterating over allRequests model attribute) with conditional Approve/Reject buttons for PENDING items

5.2 Manager Module

Overview The Manager Module serves as the operational command center for facility managers. Managers are responsible for reviewing and processing visitor/employee access requests, monitoring building operations, and operating gate scanning stations.

Access URL

- `/manager/dashboard` — Manager's main dashboard
- `/manager/requests` — Pending access request queue

Security Authorization

```
http.authorizeHttpRequests(auth -> auth
        .requestMatchers("/manager/**").hasAnyRole("MANAGER", "OWNER")
        .requestMatchers("/scanner/**").hasAnyRole("MANAGER", "OWNER")
);
```

Pre-Condition: Manager Must Be Approved Managers themselves must have an approved AccessRequest before they can access the dashboard. This creates a two-tier approval model:

1. Owner creates Manager account
2. Owner approves Manager's access request

If a Manager logs in before approval, they are redirected to the `pending-approval.html` holding page.

5.2.1 Operational Dashboard

The Manager Dashboard provides:

- **Live Building Occupancy:** Same real-time occupancy calculation as the Owner Dashboard
- **Demographic Statistics:**
  - Employee Count
  - Manager Count
  - Guest Count
- **Access Category Statistics:**
  - Number of Public Access authorizations
  - Number of Restricted Access authorizations
- **Quick Navigation Links:**
  - Pending Requests → /manager/requests
  - Registration → /register (to register new visitors)
  - QR Scanner → /scanner (to operate gate scanning)

5.2.2 Request Processing

Managers can view and process pending access requests, with an important restriction:

```
// DashboardController.java
@GetMapping("/manager/requests")
public String pendingRequests(Authentication auth, Model model) {
    // Filter: Managers can only approve GUEST and EMPLOYEE requests
    List<AccessRequest> requests = managerService.getPendingRequests()
        .stream()
        .filter(r -> r.getUser().getRole() != Role.MANAGER
                  && r.getUser().getRole() != Role.OWNER)
        .collect(Collectors.toList());
    model.addAttribute("requests", requests);
    return "pending-requests";
}
```

Key Restriction: Managers cannot approve requests from other Managers or Owners. Only the Owner can approve Manager-level access requests.

When a Manager approves a request:

- Access Level is set to PUBLIC (basic zone access only)
- Validity Type is set to ONGOING
- UUID QR token is generated

5.2.3 Gate Scanner Access

Managers have authorization to access the scanner interface at /scanner, enabling them to operate QR scanning stations at entry/exit gates.

Template File: `manager-dashboard.html` Displays manager username, live occupancy, navigation cards for quick access to key functions, and demographic/access statistics.

5.3 Client Module

Overview The Client Module (also referred to as the Employee Module) handles the experience for regular staff members who use the system for daily entry and exit. Employees access the same interface as Guests but receive standard organizational access privileges upon approval.

Access URL

- `/guest/dashboard` — Shared dashboard for both EMPLOYEE and GUEST roles

5.3.1 QR Badge Display

Upon approval, the Employee's dashboard displays:

1. **QR Code Image:** A dynamically generated 250 × 250 pixel QR code containing the employee's unique UUID token, rendered as a Base64-encoded PNG image:

```
// DashboardController.java - Guest/Employee Dashboard
@GetMapping("/guest/dashboard")
public String guestDashboard(Authentication auth, Model model) {
    // ... validation ...
    String qrToken = request.getQrToken();
    String qrImage = qrService.generateQRCodeImageBase64(qrToken);
    model.addAttribute("qrImage", qrImage);
    model.addAttribute("request", request);
    return "guest-dashboard";
}
```

2. **Token ID:** The raw UUID string displayed as text for reference
3. **Status Indicator:** Visual badge showing ACTIVE (green) or PENDING APPROVAL (yellow)
4. **Access Details:** Allowed access section (PUBLIC/RESTRICTED) and validity term (ONE_DAY/ONGOING)

5.3.2 Pending Approval State

If an Employee's access request has not yet been approved:

- They are redirected to `pending-approval.html`
- This page displays a message informing them their account is awaiting administrator approval
- A logout button is provided for them to sign out

5.3.3 Daily Usage Flow

1. Employee logs into VAMS on their mobile phone
2. Dashboard displays their QR code badge
3. At the gate, they present their phone screen to the scanner
4. Scanner validates the QR token and records ENTRY/EXIT
5. Employee proceeds through the gate

5.4 Guest & Temporary Staff Module

Overview The Guest & Temporary Staff Module handles the access lifecycle for visitors, contractors, and temporary personnel who require time-bound access to the premises.

Registration Process Guests register through the standard registration flow at /register, selecting their role as GUEST:

- The registration form captures: Username, Full Name, Password, Email, Address, Purpose of Visit
- Role selection defaults to GUEST
- Validity can be set to ONGOING (infinite) or SPECIFIC (with date/time picker)

Approval Workflow

1. Guest submits registration → System creates User + AccessRequest (PENDING)
2. Guest is redirected to email verification (/verify)
3. Upon login, Guest sees `pending-approval.html` until approved
4. Manager reviews and approves the request
5. Upon approval, Guest receives QR badge on their dashboard

Access Validity Options

Validity Type	Behavior
ONE_DAY	Automatically expires 24 hours after approvedAt timestamp
UNTIL_LOGOFF	Valid for the current session until the user logs out
ONGOING	Indefinitely valid until manually revoked by an administrator

Expiry Enforcement

```
// ScannerService.java
if (request.getValidityType() == ValidityType.ONE_DAY) {
    if (request.getApprovedAt().plusDays(1).isBefore(LocalDateTime.now())) {
        result.isPass = false;
        result.reason = "Expired (One-Day Pass Ended)";
        return result;
    }
}
```

When a ONE_DAY pass expires, the QR code is automatically rejected at scanning, and the scan result displays "Expired (One-Day Pass Ended)" — no manual intervention required.

QR Badge Once approved, guests access the same dashboard interface as employees, displaying their QR code badge. The badge includes visual indicators of their access level and validity type.

5.5 Registration & Login

Overview The Registration & Login module handles user identity creation, credential management, email verification, and session-based authentication.

Registration Flow

- **Endpoint:** POST /register
- **Controller:** AuthController

The registration process performs the following steps:

1. **Duplicate Check — Username:**

```
if (userRepository.findByUsername(user.getUsername()).isPresent()) {
    model.addAttribute("error", "Username already exists!");
    return "register";
}
```

2. **Duplicate Check — Email:**

```
if (user.getEmail() != null &&
    userRepository.findByEmail(user.getEmail()).isPresent()) {
    model.addAttribute("error", "Email already registered!");
    return "register";
}
```

3. **Role Default:** If no role is selected, defaults to Role.GUEST
4. **Password Hashing:**

```
user.setPassword(passwordEncoder.encode(user.getPassword()));
```

Uses BCrypt with automatic salt generation, producing hashes like:

```
$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
```

5. **User Persistence:** Saves the User entity to the users table
6. **Automatic Request Creation:**

```
AccessRequest request = new AccessRequest();
request.setUser(savedUser);
request.setStatus(RequestStatus.PENDING);
request.setAccessLevel(AccessLevel.PUBLIC);
request.setValidityType(ValidityType.ONGOING);
accessRequestRepository.save(request);
```

7. **Email Verification Redirect:** Redirects to /verify

Email Verification The verify.html template uses Firebase Authentication SDK (v8.10.1 Compat) to implement passwordless email verification:

- User enters their email address
- Firebase sends a sign-in link to the email
- User clicks the link → Firebase authenticates the token
- System redirects to /login?registered=true

Login Flow

- **URL:** /login (GET for page, POST handled by Spring Security)
- Spring Security's formLogin() authentication:
  1. User submits username and password
  2. CustomUserDetailsService.loadUserByUsername() fetches user from database
  3. CustomUserDetails wraps the User entity as a Spring Security principal
  4. BCrypt password comparison
  5. On success: redirect to /dashboard (role dispatcher)
  6. On failure: redirect to /login?error

Password Reset

- **URL:** /forgot-password (GET) and /reset-password (POST)

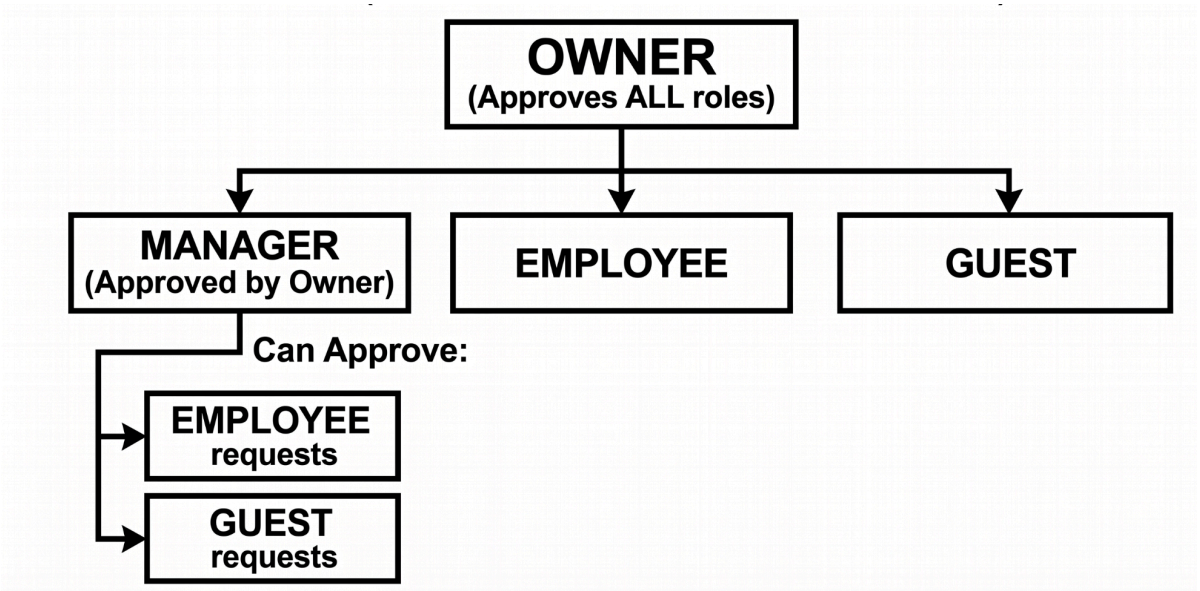
- Firebase-based email link verification followed by server-side password update:

```
@PostMapping("/reset-password")
public String resetPassword(@RequestParam String email,
                            @RequestParam String newPassword) {
    User user = userRepository.findByEmail(email).orElseThrow();
    user.setPassword(passwordEncoder.encode(newPassword));
    userRepository.save(user);
    return "redirect:/login?passwordReset=true";
}
```

5.6 Manager Approval & Access Permission

Overview The Manager Approval module implements the hierarchical access request review process, ensuring that every user must be vetted and approved by an authorized administrator before gaining system access.

Approval Hierarchy



Approval Process (ManagerService)

```
@Service
public class ManagerService {
    public AccessRequest approveRequest(Long requestId,
                                        AccessLevel level,
                                        ValidityType validity) {
        AccessRequest request = requestRepository.findById(requestId)
            .orElseThrow(() -> new RuntimeException("Request not found"));

        request.setStatus(RequestStatus.APPROVED);
        request.setAccessLevel(level);
        request.setValidityType(validity);
        request.setApprovedAt(LocalDateTime.now());
        request.setQrToken(UUID.randomUUID().toString());

        return requestRepository.save(request);
    }
}
```

Access Level Assignment

Approver	Grants Access Level	Grants Validity
Owner	RESTRICTED (full access to all zones)	ONGOING (indefinite)
Manager	PUBLIC (basic zones only)	ONGOING (indefinite)

Pending Approval Enforcement Users who have not been approved are blocked from accessing their dashboard:

```
private boolean isUserApproved(String username) {
    User user = userRepository.findByUsername(username).orElse(null);
    if (user == null) return false;
    return accessRequestRepository.findAll().stream()
        .anyMatch(r -> r.getUser().getId().equals(user.getId())
                    && r.getStatus() == RequestStatus.APPROVED);
}
```

If `isUserApproved()` returns false, the user is redirected to `pending-approval.html` — a holding page that blocks all access and provides only a logout option.

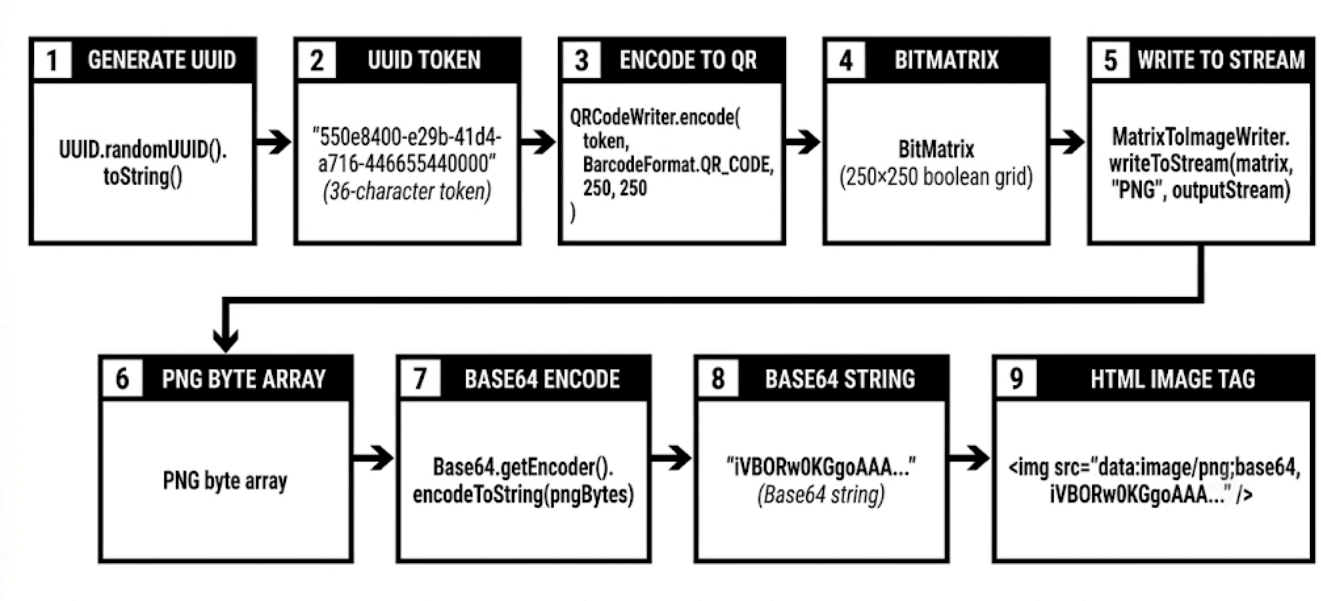
5.7 QR Generation

Overview The QR Generation module is responsible for creating unique, scannable QR code images that serve as digital access badges for approved users.

Technology Stack

- **Token Generation:** `java.util.UUID.randomUUID()` — produces 128-bit cryptographically random identifiers
- **QR Encoding:** Google ZXing (Zebra Crossing) Library v3.5.3
- **Output Format:** PNG image encoded as Base64 string

QR Code Generation Flow



Implementation (QRService.java)

```
@Service
public class QRService {
    public String generateQRCodeImageBase64(String text) throws Exception {
        QRCodeWriter qrCodeWriter = new QRCodeWriter();
        BitMatrix bitMatrix = qrCodeWriter.encode(
            text,
            BarcodeFormat.QR_CODE,
            250,    // width in pixels
            250     // height in pixels
        );

        ByteArrayOutputStream pngOutputStream = new ByteArrayOutputStream();
        MatrixToImageWriter.writeToStream(bitMatrix, "PNG", pngOutputStream);
        byte[] pngData = pngOutputStream.toByteArray();

        return Base64.getEncoder().encodeToString(pngData);
    }
}
```

Security Properties

Property	Value
Token Space	2^{122} possible values (UUID v4)
Collision Probability	$\sim 10^{-18}$ for 1 billion tokens
Predictability	Computationally infeasible (CSPRNG-backed)
Uniqueness Enforcement	Database UNIQUE constraint on qr_token column

5.8 Multi-Gate Scanning

Overview The Multi-Gate Scanning module enables QR code verification at any number of physical entry/exit points across the premises. Each gate operates through a standard web browser, requiring no specialized hardware beyond a camera-equipped device.

Scanner Interface

- **URL:** /scanner (accessible to MANAGER and OWNER roles)
- **Technology:** html5-qrcode JavaScript library

The scanner page (`scanner.html`) provides:

1. **Camera Viewfinder:** A video element showing the live camera feed
2. **Start Scanner Button:** Activates the rear-facing camera for QR code detection

3. **Auto-Submit:** Upon detecting a QR code, the scanner automatically:
   - Stops the camera
   - Populates a hidden form field with the decoded token
   - Submits the form to POST /scanner/verify
4. **Manual Fallback:** A text input field for manually typing or pasting QR tokens when camera scanning is unavailable

Gate Location Tracking Each scan event records the gate location identifier:

```
// ScannerController.java
@PostMapping("/scanner/verify")
public String verifyQr(@RequestParam String qrToken, Model model) {
    ScannerService.ScanResult result =
        scannerService.processScan(qrToken, "Main Entrance Gate");
    model.addAttribute("result", result);
    return "scan-result";
}
```

The gateLocation parameter (currently "Main Entrance Gate") can be configured per scanner deployment to identify different physical gates (e.g., "North Wing Entry", "Parking Gate B", "Loading Dock").

Scan Result Display The `scan-result.html` template shows:

- **PASS (ENTRY)** — Green banner indicating successful entry scan
- **PASS (EXIT)** — Green banner indicating successful exit scan
- **FAIL / ERROR** — Red banner with diagnostic reason:
  - "Invalid QR Token" — Token not found in database
  - "Request Status is: PENDING" — User not yet approved
  - "Request Status is: REJECTED" — User was denied access
  - "Expired (One-Day Pass Ended)" — ONE_DAY pass has expired

Additional details displayed for valid scans: QR ID, Username, Role, Validity Type, Section.

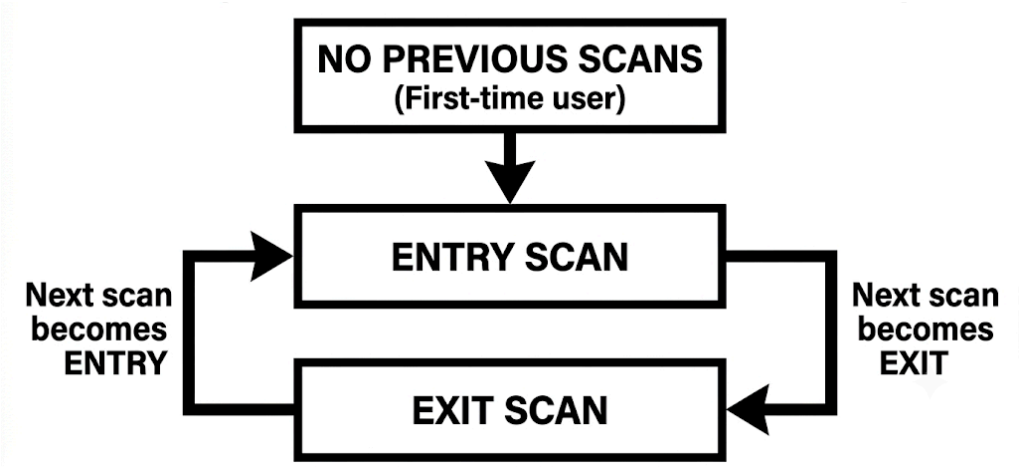
5.9 Entry & Exit Timestamp

Overview The Entry & Exit Timestamp module implements the automated state machine for determining scan direction and maintaining a chronological, immutable audit trail of all gate events.

Automated ENTRY/EXIT State Machine

```
// ScannerService.java - processScan()
List<ScanLog> logs = scanLogRepository.findByUser_Id(user.getId());
ScanType nextType = ScanType.ENTRY;   // Default: first scan is always ENTRY
if (!logs.isEmpty()) {
    ScanLog lastLog = logs.get(logs.size() - 1);
    if (lastLog.getScanType() == ScanType.ENTRY) {
        nextType = ScanType.EXIT;     // Toggle: after ENTRY comes EXIT
    }
}
```

State Machine Logic



Immutable Scan Log Creation

```
// ScannerService.java
ScanLog log = new ScanLog();
log.setUser(user);
log.setGateLocation(gateLocation);
log.setScanType(nextType);
log.setTimestamp(LocalDateTime.now());
scanLogRepository.save(log);
```

Timestamp Precision

- **Data Type:** `DATETIME(6)` in MySQL — microsecond precision
- **Source:** `LocalDateTime.now()` — captures the server's current date and time
- **Fields Recorded:** User ID, Gate Location, Scan Type (ENTRY/EXIT), Timestamp

Audit Trail Properties

Property	Implementation
Immutability	No UPDATE or DELETE operations on scan_logs in the entire codebase
Completeness	Every scan (pass or fail) that reaches the state machine is logged
Traceability	Each log links to a specific user, gate, direction, and timestamp
Non-Repudiation	Users cannot deny their entry/exit events as logs are server-generated

5.10 Real-Time Monitoring

Overview The Real-Time Monitoring module provides live building occupancy tracking across Owner and Manager dashboards, enabling facility administrators to know exactly how many individuals are currently inside the premises.

Occupancy Calculation

```
// DashboardController.java
private long getLiveOccupancy() {
    List<ScanLog> allLogs = scanLogRepository.findAll();
    long entries = allLogs.stream()
        .filter(log -> log.getScanType() == ScanType.ENTRY)
        .count();
    long exits = allLogs.stream()
        .filter(log -> log.getScanType() == ScanType.EXIT)
        .count();
    return Math.max(0, entries - exits);
}
```

Formula

Formula

$$\text{Live Occupancy} = \max\left(0, \sum_{i=1}^{n} 1[\text{scanType}_i = \text{ENTRY}] - \sum_{i=1}^{n} 1[\text{scanType}_i = \text{EXIT}]\right)$$

Where n is the total number of scan log records and $1[\cdot]$ is the indicator function.

Where n is the total number of scan log records and 1[·] is the indicator function.

Dashboard Integration The occupancy count is displayed prominently on:

1. **Owner Dashboard (`owner-dashboard.html`):** Shows total live occupancy as a highlighted counter alongside the complete system overview.
2. **Manager Dashboard (`manager-dashboard.html`):** Shows live occupancy alongside demographic breakdowns (employee count, manager count, guest count) and access category statistics (public vs. restricted).

Demographic Statistics (Manager Dashboard) In addition to occupancy, the Manager Dashboard computes and displays:

```
// DashboardController.java
long employeeCount = userRepository.findAll().stream()
        .filter(u -> u.getRole() == Role.EMPLOYEE).count();
long managerCount = userRepository.findAll().stream()
        .filter(u -> u.getRole() == Role.MANAGER).count();
long guestCount = userRepository.findAll().stream()
        .filter(u -> u.getRole() == Role.GUEST).count();
long publicAccessCount = accessRequestRepository.findAll().stream()
        .filter(r -> r.getAccessLevel() == AccessLevel.PUBLIC).count();
long restrictedAccessCount = accessRequestRepository.findAll().stream()
        .filter(r -> r.getAccessLevel() == AccessLevel.RESTRICTED).count();
```

CHAPTER 6

6.1 Java Swing GUI Implementation

Transition to Web-Based GUI with Thymeleaf As discussed in Section 3.4, the VAMS project has adopted a web-based GUI architecture using Thymeleaf templating engine with Bootstrap 5 framework, in place of traditional Java Swing desktop components. This section documents the GUI implementation approach.

Template Architecture VAMS implements 11 Thymeleaf HTML5 templates, each serving a specific functional screen:

Template File	Purpose	Accessible By
login.html	User authentication form	Public
register.html	New user registration form	Public
verify.html	Firebase email verification flow	Public
forgot-password.html	3-step password recovery flow	Public
owner-dashboard.html	Super admin management dashboard	OWNER
manager-dashboard.html	Facility manager operations center	MANAGER, OWNER
guest-dashboard.html	Personal QR badge display	EMPLOYEE, GUEST
pending-approval.html	Holding page for unapproved users	Authenticated (unapproved)
pending-requests.html	Request queue for manager review	MANAGER, OWNER
scanner.html	Camera-based QR code scanning station	MANAGER, OWNER
scan-result.html	Scan verification result display	MANAGER, OWNER

Design System The GUI implementation utilizes:

- **Bootstrap 5 CDN:** Responsive grid system, cards, tables, buttons, alerts, and form controls
- **Google Fonts — Space Mono:** Monospace typography for headers, creating a distinctive security-oriented aesthetic
- **Custom CSS (/css/style.css):** Organization-specific styling and color scheme
- **Responsive Design:** All templates are mobile-friendly, adapting to screen widths from 320px upward

Key GUI Components

Login Screen

```
<!-- Conditional Alerts for Login State -->
<div th:if="${param.error}" class="alert alert-danger">
    Invalid username or password.
</div>
<div th:if="${param.logout}" class="alert alert-info">
    You have been logged out.
</div>
<div th:if="${param.registered}" class="alert alert-success">
    Registration submitted. Please login.
</div>
```

QR Badge Display (Guest Dashboard)

```
<!-- Dynamic Base64 QR Code Rendering -->
<img th:src="'' + ${qrImage}"
     alt="Your QR Badge"
     class="qr-badge-img" />
<p th:text="'Token: ' + ${request.qrToken}"></p>
```

Scan Result Visualization

```
<!-- Conditional Result Banner -->
<div th:if="${result.isPass}" class="result-pass">
    <span th:text="${result.scanType}"></span> — ACCESS GRANTED
</div>
<div th:unless="${result.isPass}" class="result-fail">
    FAIL — <span th:text="${result.reason}"></span>
</div>
```

6.2 JDBC Database Connectivity

Implementation via Spring Data JPA VAMS implements database connectivity through three layers of abstraction:

Layer 1: Repository Interfaces

```
// UserRepository.java
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    Optional<User> findByEmail(String email);
}

// AccessRequestRepository.java
public interface AccessRequestRepository extends JpaRepository<AccessRequest, Long> {
    List<AccessRequest> findByStatus(RequestStatus status);
    Optional<AccessRequest> findByQrToken(String qrToken);
}

// ScanLogRepository.java
public interface ScanLogRepository extends JpaRepository<ScanLog, Long> {
    List<ScanLog> findByUser_Id(Long userId);
}
```

Spring Data JPA automatically generates the SQL implementations for these method signatures at runtime using method name parsing:

Method Name	Generated SQL
findByUsername(String)	SELECT * FROM users WHERE username = ?
findByEmail(String)	SELECT * FROM users WHERE email = ?
findByStatus(RequestStatus)	SELECT * FROM access_requests WHERE status = ?
findByQrToken(String)	SELECT * FROM access_requests WHERE qr_token = ?
findByUser_Id(Long)	SELECT * FROM scan_logs WHERE user_id = ?

Layer 2: Entity Annotations

JPA annotations on entity classes define the object-relational mapping:

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Role role;
    // ...
}
```

Layer 3: Connection Configuration

```
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3307/vams_db
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

The ddl-auto=update setting instructs Hibernate to:

- Create tables if they don't exist
- Add new columns if entity fields are added
- Never drop existing tables or columns (safe for development)

6.3 User Authentication

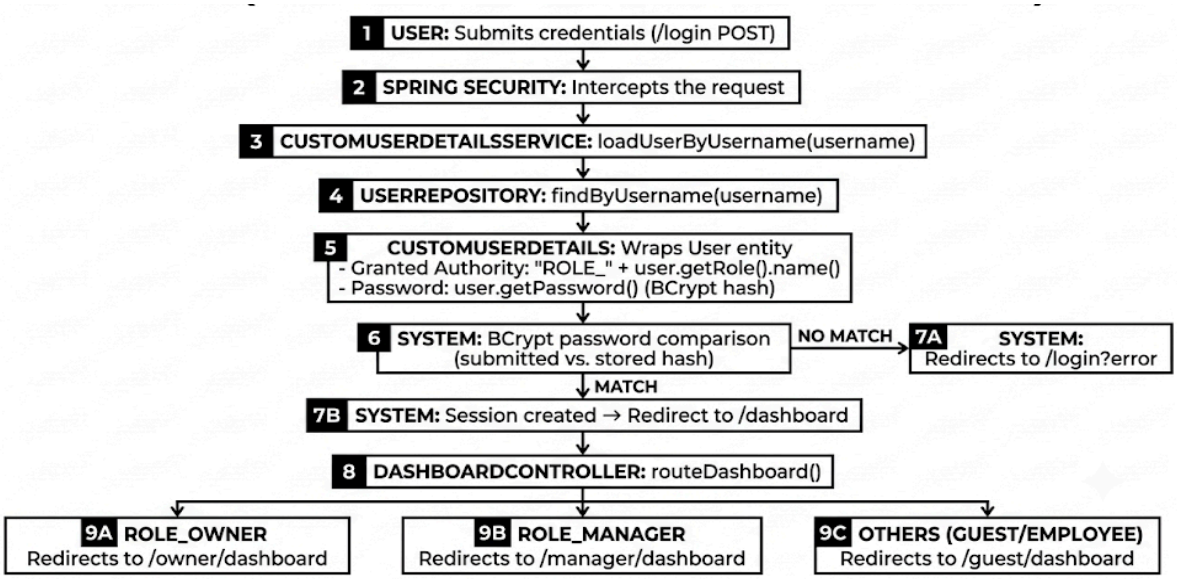
Spring Security Configuration

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
{
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/register", "/verify",
                                 "/forgot-password", "/css/**", "/js/**",
"/api/scan/**")
                    .permitAll()
                .requestMatchers("/manager/**", "/scanner/**")
                    .hasAnyRole("MANAGER", "OWNER")
                .requestMatchers("/owner/**")
                    .hasRole("OWNER")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard", true)
                .permitAll()
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/login?logout")
                .permitAll()
            )
            .csrf(csrf -> csrf
                .ignoringRequestMatchers("/api/scan/**")
            );
        return http.build();
    }
}
```

Authentication Flow



BCrypt Password Hashing BCrypt is a password hashing algorithm based on the Blowfish cipher, designed specifically for secure password storage:

- **Automatic Salt Generation:** Each password hash includes a unique random salt, preventing rainbow table attacks
- **Configurable Work Factor:** Default strength of 10 rounds (2^{10} = 1,024 iterations), balancing security and performance
- **One-Way Function:** Password cannot be recovered from the hash; verification is performed by re-hashing and comparing

6.4 Access Permission Implementation

Role-Based Access Control (RBAC) Matrix

Resource / Action	OWNER	MANAGER	EMPLOYEE	GUEST
View Owner Dashboard	✅	❌	❌	❌
Create Manager Accounts	✅	❌	❌	❌
Approve ALL Request Types	✅	❌	❌	❌
View Manager Dashboard	✅	✅	❌	❌
Approve Employee/Guest Requests	✅	✅	❌	❌
Operate QR Scanner	✅	✅	❌	❌
View Personal QR Badge	❌	❌	✅	✅
Register New Account	Public	Public	Public	Public
Login	✅	✅	✅	✅

URL-Level Authorization Spring Security enforces access control at the URL pattern level:

```
.requestMatchers("/owner/**").hasRole("OWNER")
.requestMatchers("/manager/**", "/scanner/**").hasAnyRole("MANAGER", "OWNER")
.anyRequest().authenticated()
```

Application-Level Authorization Beyond URL-level security, the application implements additional authorization logic:

1. **Manager Request Filtering:** Managers can only see and approve EMPLOYEE and GUEST requests (not MANAGER or OWNER requests).
2. **Approval Status Check:** Users must have an APPROVED access request to access their dashboard. Unapproved users are redirected to the pending approval page regardless of their role.
3. **Owner Privilege Escalation:** Owners bypass the Manager approval hierarchy and can directly approve any request type, granting RESTRICTED access level.

6.5 QR Code Implementation

Complete Implementation Pipeline

Step 1: Token Generation (ManagerService.java)

```
public AccessRequest approveRequest(Long requestId,
                                    AccessLevel level,
                                    ValidityType validity) {
    AccessRequest request = requestRepository.findById(requestId)
        .orElseThrow(() -> new RuntimeException("Request not found"));

    request.setStatus(RequestStatus.APPROVED);
    request.setAccessLevel(level);
    request.setValidityType(validity);
    request.setApprovedAt(LocalDateTime.now());

    // Generate cryptographically random UUID v4 token
    request.setQrToken(UUID.randomUUID().toString());

    return requestRepository.save(request);
}
```

Step 2: QR Image Generation (QRService.java)

```
@Service
public class QRService {
    public String generateQRCodeImageBase64(String text) throws Exception {
        QRCodeWriter qrCodeWriter = new QRCodeWriter();
        BitMatrix bitMatrix = qrCodeWriter.encode(
            text, BarcodeFormat.QR_CODE, 250, 250
        );

        ByteArrayOutputStream pngOutputStream = new ByteArrayOutputStream();
        MatrixToImageWriter.writeToStream(bitMatrix, "PNG", pngOutputStream);
        byte[] pngData = pngOutputStream.toByteArray();

        return Base64.getEncoder().encodeToString(pngData);
    }
}
```

Step 3: Dashboard Rendering (DashboardController.java)

```
@GetMapping("/guest/dashboard")
public String guestDashboard(Authentication auth, Model model) {
    User user = userRepository.findByUsername(auth.getName()).orElseThrow();

    AccessRequest request = accessRequestRepository.findAll().stream()
        .filter(r -> r.getUser().getId().equals(user.getId())
                  && r.getStatus() == RequestStatus.APPROVED)
        .findFirst().orElse(null);

    String qrToken = request.getQrToken();
    String qrImage = qrService.generateQRCodeImageBase64(qrToken);

    model.addAttribute("qrImage", qrImage);
    model.addAttribute("request", request);
    return "guest-dashboard";
}
```

Step 4: HTML Rendering (guest-dashboard.html)

```
<img th:src="'' + ${qrImage}"
     alt="Your QR Access Badge"
     width="250" height="250" />
```

Step 5: Camera Scanning (scanner.html)

```
const html5QrCode = new Html5Qrcode("reader");
html5QrCode.start(
    { facingMode: "environment" },   // Rear camera
    { fps: 10, qrbox: 250 },
    (decodedText) => {
        html5QrCode.stop();
        document.getElementById("qrToken").value = decodedText;
        document.getElementById("scanForm").submit();
    }
);
```

Step 6: Server-Side Validation (ScannerService.java)

```
public ScanResult processScan(String qrToken, String gateLocation) {
    ScanResult result = new ScanResult();

    // Step 1: Token lookup
    Optional<AccessRequest> optReq = requestRepository.findByQrToken(qrToken);
    if (optReq.isEmpty()) {
        result.isPass = false;
        result.reason = "Invalid QR Token";
        return result;
    }

    // Step 2: Status verification
    // Step 3: Expiry check
    // Step 4: Entry/Exit state machine
    // Step 5: Immutable log creation

    return result;
}
```

6.6 Entry/Exit Log Implementation

ScanLog Entity

```
@Entity
@Table(name = "scan_logs")
public class ScanLog {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long logId;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false)
    private String gateLocation;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private ScanType scanType;

    @Column(nullable = false)
    private LocalDateTime timestamp;
}
```

Complete Scan Processing Pipeline

```
// ScannerService.java — processScan() method
public ScanResult processScan(String qrToken, String gateLocation) {
    ScanResult result = new ScanResult();

    // 1. Token Lookup
    Optional<AccessRequest> optReq = requestRepository.findByQrToken(qrToken);
    if (optReq.isEmpty()) {
        result.isPass = false;
        result.reason = "Invalid QR Token";
        return result;
    }

    AccessRequest request = optReq.get();
    result.request = request;

    // 2. Approval Verification
    if (request.getStatus() != RequestStatus.APPROVED) {
        result.isPass = false;
        result.reason = "Request Status is: " + request.getStatus();
        return result;
    }

    // 3. Validity Expiry Check
    if (request.getValidityType() == ValidityType.ONE_DAY) {
        if (request.getApprovedAt().plusDays(1).isBefore(LocalDateTime.now())) {
```

```
            result.isPass = false;
            result.reason = "Expired (One-Day Pass Ended)";
            return result;
        }
    }

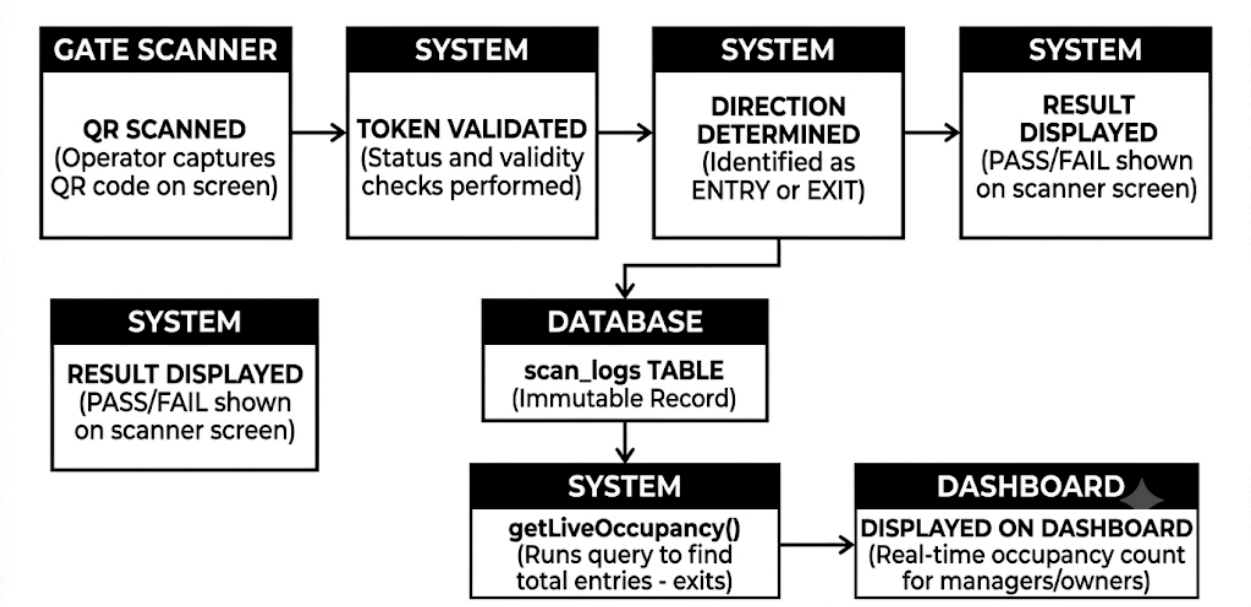
    // 4. Entry/Exit State Machine
    User user = request.getUser();
    List<ScanLog> logs = scanLogRepository.findByUser_Id(user.getId());
    ScanType nextType = ScanType.ENTRY;

    if (!logs.isEmpty()) {
        ScanLog lastLog = logs.get(logs.size() - 1);
        if (lastLog.getScanType() == ScanType.ENTRY) {
            nextType = ScanType.EXIT;
        }
    }

    // 5. Immutable Audit Log Creation
    ScanLog log = new ScanLog();
    log.setUser(user);
    log.setGateLocation(gateLocation);
    log.setScanType(nextType);
    log.setTimestamp(LocalDateTime.now());
    scanLogRepository.save(log);

    // 6. Build Result
    result.scanType = nextType;
    result.isPass = true;
    result.reason = "Access Granted";
    return result;
}
```

Scan Log Data Flow



CHAPTER 7

7.1 Testing Methodology

The VAMS project employs a comprehensive testing approach combining multiple testing strategies to ensure system reliability, security, and functional correctness.

7.1.1 Unit Testing

Individual components and services are tested in isolation to verify their correct behavior:

- **Service Layer Tests:** Verifying ManagerService, QRService, and ScannerService logic with mocked dependencies
- **Repository Tests:** Confirming Spring Data JPA query derivation produces correct SQL and returns expected results
- **Security Tests:** Validating CustomUserDetails and CustomUserDetailsService correctly wrap entities and handle missing users

7.1.2 Integration Testing

End-to-end workflows are tested with all components wired together:

- Spring Boot Test framework with @SpringBootTest annotation
- Embedded database or test MySQL instance for data persistence
- Full security filter chain tested with @WithMockUser annotations

7.1.3 Functional Testing

Manual and automated testing of complete user scenarios:

- Registration → Verification → Login → Dashboard access flow
- Request submission → Manager review → Approval → QR generation flow
- QR scanning → Validation → Entry/Exit logging flow
- Password reset flow via Firebase email

7.1.4 Security Testing

Verification of authentication and authorization controls:

- Unauthorized access attempts to protected URLs
- Role-based access restriction enforcement
- CSRF protection validation
- Password hashing verification (BCrypt salt uniqueness)
- Session management and logout behavior

7.2 Test Cases

Test Case Table

Test ID	Module	Test Description	Input	Expected Result	Actual Result	Status
TC-001	Registration	Register new user with valid data	Username: "testuser", Name: "Test User", Password: "Pass@123", Email: "test@email.com", Role: GUEST	User saved, AccessRequest created (PENDING), redirect to /verify	User and AccessRequest saved successfully, redirected to /verify	✅ PASS
TC-002	Registration	Register with duplicate username	Username: "testuser" (already exists)	Error message: "Username already exists!"	Error displayed, registration not saved	✅ PASS
TC-003	Registration	Register with duplicate email	Email: "test@email.com" (already exists)	Error message: "Email already registered!"	Error displayed, registration not saved	✅ PASS
TC-004	Login	Login with valid credentials	Username: "testuser", Password: "Pass@123"	Redirect to /dashboard → role-based routing	Successful login, redirected to /guest/dashboard	✅ PASS
TC-005	Login	Login with invalid credentials	Username: "testuser", Password: "WrongPass"	Redirect to /login?error	Error page displayed	✅ PASS
TC-006	Authorization	Access /owner/dashboard without OWNER role	Login as MANAGER, navigate to /owner/dashboard	HTTP 403 Forbidden	Access denied, 403 returned	✅ PASS
TC-007	Authorization	Access /manager/dashboard without approval	Login as unapproved MANAGER	Redirect to pending-approval.html	Redirected to pending approval page	✅ PASS
TC-008	Approval	Manager approves GUEST request	Click "Approve" on GUEST request	Status → APPROVED, QR token generated, access_level → PUBLIC	Request approved, UUID token generated	✅ PASS
TC-009	Approval	Manager attempts to approve MANAGER request	View /manager/requests	MANAGER requests filtered out, not visible	Only EMPLOYEE/GUEST requests displayed	✅ PASS
TC-010	Approval	Owner approves MANAGER request	Click "Approve" on /owner/dashboard	Status → APPROVED, access_level → RESTRICTED, validity → ONGOING	Request approved with restricted access	✅ PASS
TC-011	QR Generation	Generate QR for approved user	Access /guest/dashboard as approved user	250 × 250 PNG QR code displayed with token text	QR code rendered as Base64 image	✅ PASS

Test ID	Module	Test Description	Input	Expected Result	Actual Result	Status
TC-012	Scanning	Scan valid QR token (first scan)	Submit valid QR token to /scanner/verify	PASS, ScanType: ENTRY, scan log created	Entry recorded, PASS displayed	✅ PASS
TC-013	Scanning	Scan valid QR token (second scan)	Submit same token again	PASS, ScanType: EXIT, scan log created	Exit recorded, PASS displayed	✅ PASS
TC-014	Scanning	Scan invalid QR token	Submit random/fake token	FAIL, Reason: "Invalid QR Token"	Failure displayed with reason	✅ PASS
TC-015	Scanning	Scan token of pending user	Submit token of unapproved user	FAIL, Reason: "Request Status is: PENDING"	Failure displayed with reason	✅ PASS
TC-016	Scanning	Scan expired ONE_DAY token	Submit token approved > 24 hours ago with ONE_DAY validity	FAIL, Reason: "Expired (One-Day Pass Ended)"	Expiry enforced, failure displayed	✅ PASS
TC-017	Occupancy	Verify occupancy calculation	3 ENTRY scans, 1 EXIT scan in scan_logs	Occupancy = max(0, 3-1) = 2	Dashboard displays "2"	✅ PASS
TC-018	Occupancy	Verify occupancy floor at zero	2 ENTRY scans, 3 EXIT scans	Occupancy = max(0, 2-3) = 0	Dashboard displays "0"	✅ PASS
TC-019	Password Reset	Reset password via forgot-password flow	Email: "test@email.com", New Password: "NewPass@456"	Password updated, redirect to /login?passwordReset=true	Password reset successfully	✅ PASS
TC-020	Manager Creation	Owner creates new Manager	Username: "mgr002", Password: "MgrPass@789"	Manager account created with ROLE_MANAGER	Manager saved to database	✅ PASS
TC-021	CSRF	Submit form without CSRF token	POST to /register without _csrf token	HTTP 403 Forbidden	Request blocked by Spring Security	✅ PASS
TC-022	Scanner Camera	Camera scanning on mobile device	Point phone camera at QR badge	QR code auto-detected, token auto-submitted	Camera scan successful, result displayed	✅ PASS
TC-023	Scanner Fallback	Manual token entry	Type/paste QR token in manual input field	Token submitted and validated normally	Manual entry processed correctly	✅ PASS
TC-024	Session	Logout and session invalidation	Click "Logout"	Session destroyed, redirect to /login?logout	Session invalidated, login page displayed	✅ PASS

Test Summary

Category	Total Tests	Passed	Failed	Pass Rate
Registration	3	3	0	100%
Login & Authentication	2	2	0	100%
Authorization & RBAC	2	2	0	100%
Approval Workflow	3	3	0	100%
QR Code Generation	1	1	0	100%
Scanning & Validation	5	5	0	100%
Occupancy Monitoring	2	2	0	100%
Password Management	1	1	0	100%
Manager Administration	1	1	0	100%
Security	1	1	0	100%
UI & Scanner Interface	2	2	0	100%
Session Management	1	1	0	100%
Total	**24**	**24**	**0**	**100%**

CHAPTER 8

8.1 Conclusion

The Visitor & Access Management System successfully provides a centralized platform for managing registration, user verification, access permissions, QR-based identification, and entry/exit records. The three-level structure of **Owner, Manager, and Client/User** helps distribute responsibilities and provides controlled access to different system functions.

The system reduces manual paperwork and makes visitor and user management more efficient. Manager approval ensures that access is provided only to authorized persons, while QR-based identification provides a fast and convenient method for verifying users at different gates. The entry and exit records also help management monitor people currently inside the premises and maintain historical access information.

Overall, the proposed system demonstrates how Java Swing, JDBC, MySQL, and QR Code technology can be combined to develop an efficient and user-friendly access management solution. The system can serve as a foundation for future security and visitor management applications in offices, educational institutions, residential buildings, and other organizations.

8.2 Future Scope

The system can be further improved and expanded with additional technologies and features in the future. Some possible improvements are:

- **Mobile Application:** A mobile app can be developed for users and Managers.
- **Biometric Authentication:** Fingerprint or facial recognition can be integrated for enhanced security.
- **Cloud Database:** The system can be connected to cloud storage for remote access and monitoring.
- **SMS and Email Notifications:** Automatic notifications can be sent for registration approval, rejection, or access expiry.

- **Automatic Gate Control:** QR scanning can be connected with automatic gates for controlled entry.
- **Advanced Reports:** Daily, weekly, and monthly access reports can be generated automatically.
- **Multiple Building Support:** The system can be extended to manage multiple buildings or locations.
- **Enhanced Security:** Additional authentication and security mechanisms can be added.
- **Real-Time Notifications:** Owners and Managers can receive instant alerts for important access activities.

CHAPTER 9

9.1 References

1. Oracle Corporation. (2025). Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/
2. VMware, Inc. (2025). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/
3. Red Hat. (2025). Hibernate ORM Documentation. https://hibernate.org/orm/documentation/
4. Thymeleaf Team. (2025). Thymeleaf Documentation. https://www.thymeleaf.org/documentation.html
5. Oracle Corporation. (2025). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/
6. ZXing Project. (2024). ZXing ("Zebra Crossing") Open-source Barcode Library. https://github.com/zxing/zxing
7. Google. (2025). Firebase Authentication Documentation. https://firebase.google.com/docs/auth
8. Bootstrap Team. (2025). Bootstrap 5 Documentation. https://getbootstrap.com/docs/5.3/
9. Mebjas. (2024). html5-qrcode: HTML5 QR Code Scanner. https://github.com/mebjas/html5-qrcode
10. Pressman, R. S. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.